

zuolan v5 FPGA 与 STM32 双核信号处理系统设计说明

架构、接口、采集时序与算法推导

左岚

2026 年 7 月

摘要

本文说明基于 STM32F429IGTx 与 EP4CE10F17C8 的双核信号处理系统。系统使用 FPGA 完成双路 AD9226 并行采集、比较器门控计数、FMC 寄存器接口及双路 AD9764 波形输出，使用 STM32 完成工作模式控制、自适应测频、幅度与相位测量、Hann 窗频谱分析及 THD、THD+N、SINAD 计算。本文给出模块连接、时钟域关系、FMC 地址、双 ADC 固定配对方案以及主要算法的数学推导。文中性能数据只对应当前代码版本和已执行测试，不扩展为未经校准的整机指标。

关键词：STM32F429；EP4CE10；AD9226；AD9764；FMC；FFT；相位测量

目录

1	系统概述	4
1.1	功能边界	4
1.2	主要数据路径	4
1.3	设计目标与边界约束	4
1.4	硬件组成与信号形态	5
1.5	时钟域、复位域与跨域边界	5
1.6	一次完整测量的事务边界	5
1.7	数据格式与符号约定	6
2	FPGA 设计	7
2.1	模块与时钟域	7
2.2	顶层模块的职责与连接策略	7
2.2.1	寄存器线网解包	7
2.2.2	子模块实例化与双路独立性	8
2.3	FMC 地址与直接读回	9
2.3.1	FMC 事务译码	10
2.3.2	为什么 FIFO 必须直接读回	10

2.4	ADC 采集与固定配对	11
2.4.1	上升沿检测与写请求延迟	11
2.4.2	128/1024 点截止条件	12
2.5	采样 NCO 推导	12
2.5.1	NCO 关键代码解析	13
2.6	DA 地址与幅度流水	14
2.6.1	DA 频率、相位与 ROM 地址	14
2.6.2	Q24 幅度缩放流水	15
2.7	比较器门控计数	17
3	STM32 软件设计	18
3.1	软件分层	18
3.2	启动流程与模块初始化顺序	18
3.3	非抢占式调度器	20
3.4	FMC 内存映射与控制位 API	22
3.5	Key4 测量流程	25
3.5.1	按键边沿检测与回调表	25
3.5.2	频率来源选择与交叉校验	27
3.5.3	测量模式分流	29
3.6	自适应采样率	29
3.6.1	正式采样率的代码实现	30
3.6.2	三档粗测与过零精测	31
3.7	比较器门控测频	33
3.8	DA 输出的软件影子配置	34
4	数字信号处理算法	36
4.1	ADC 电压与峰峰值	36
4.1.1	FIFO 读回和有效性防线	36
4.2	过零频率估计	38
4.3	Hann 窗、FFT 与幅值恢复	40
4.3.1	CMSIS-DSP 缓冲组织与归一化	40
4.3.2	三点抛物线峰值插值	41
4.4	双路基波相位	43
4.4.1	双路采集与相位补偿代码	43
4.5	THD、THD+N 与 SINAD	45
5	接口与引脚摘要	47

5.1	FPGA 外部接口	47
5.2	STM32 外设接口	47
6	构建与验证	47
6.1	构建环境	47
6.2	轻量回归	48
6.3	验证层次与证据边界	48
6.4	常见故障与定位顺序	48
6.5	日志解读	49
7	结论与维护约束	49
A	Key4 默认配置	50
B	术语与符号表	50
C	关键源码索引	50
D	相关文档	51

1 系统概述

1.1 功能边界

FPGA 负责确定性时序与并行数据路径，STM32 负责控制决策和浮点算法。外部比较器输入只用于独立测频，不参与 ADC 并行数据采样。STM32 与 FPGA 之间通过 16 位异步 FMC 复用地址/数据总线通信。

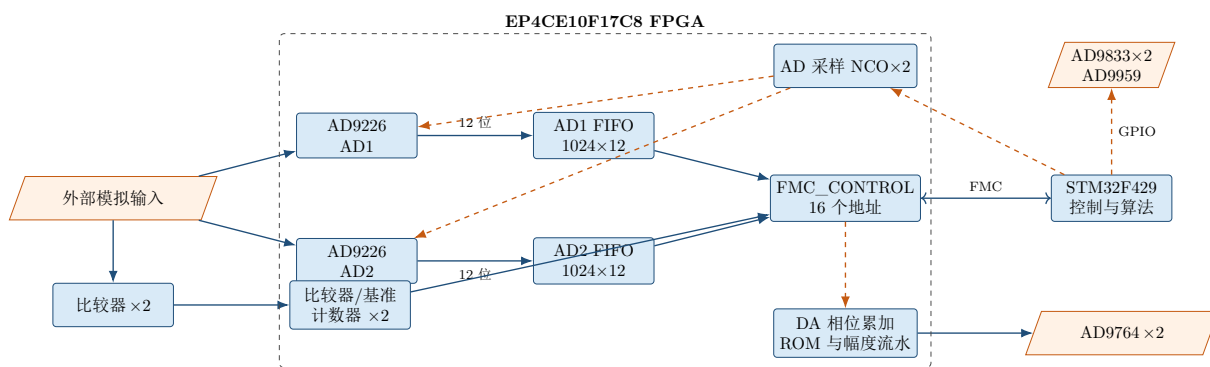


图 1: 系统数据流与控制边界

1.2 主要数据路径

- 1) ADC 路径: AD9226 并行码 → FPGA FIFO → FMC → STM32。
- 2) 比较器路径: 外部比较器脉冲 → FPGA 门控计数 → FMC → STM32。
- 3) FPGA DA 路径: STM32 参数 → FMC → 相位累加/ROM/幅度流水 → AD9764。
- 4) 外部 DDS 路径: STM32 GPIO 串行接口 → AD9833 或 AD9959。

1.3 设计目标与边界约束

本系统不是将所有功能集中到单一处理器，而是按「时序确定性」分工。FPGA 处理对时钟边沿、并行总线和固定流水线敏感的部分；STM32 处理测量策略、浮点计算、人机交互和设备配置。这一划分形成如下约束：

- 1) ADC 采样时钟、FIFO 写请求和 DA 数据流必须由 FPGA 生成，不依赖软件轮询精度。
- 2) STM32 只通过 FMC 寄存器契约改变 FPGA 状态，不直接参与 FPGA 内部时钟门控。
- 3) 双路相位测量使用固定样本配对和显式时差补偿，不使用幅度修正掩盖采样错位。
- 4) 文档中的频率、时差和精度结论必须能追溯到当前源码或已执行的实板测试。

1.4 硬件组成与信号形态

表 1: 系统主要器件与信号形态

器件/资源	与主控的信号	作用与边界
AD9226×2	12 位并行数据 + 采样时钟	将两路模拟信号转为偏移二进制码，采样时钟由独立 NCO 输出。
AD9764×2	14 位并行数据 + 固定采样时钟	接收 FPGA 的 ROM 波形和幅度缩放结果，数据更新与采样沿错开。
EP4CE10F17C8	GPIO/FMC/并行数据	承担实时数据面：NCO、FIFO、计数器、寄存器和 DA 流水线。
STM32F429IGTx	16 位异步 FMC+GPIO+USART	承担控制面和算法面：配置、采集管理、FFT、相位和交互。
比较器 ×2	独立脉冲输入	只用于门控测频，不与 AD9226 并行码采样混用。
AD9833/AD9959	GPIO 软件串行接口	作为外部 DDS/调制信号源，独立于 FPGA 内部 AD9764 波形路径。

1.5 时钟域、复位域与跨域边界

表 2: 主要时钟域与跨域约束

时钟/事件	关联模块	主要约束
clk_base 150 MHz	FMC、ADC NCO、FIFO 写端、基准计数	ADC 采样沿在此域检测，写请求延后一个周期使数据稳定。
da_data_clock 125 MHz	DA 相位累加、ROM、幅度流水	配置只在 DA 停止时更新，避免多位参数中途改变。
da_capture_clock	AD9764 采样输出	与数据时钟同频并延时，为 DAC 建立/保持时间留出余量。
fmc_read_enable	FIFO 读端	异步读事务同时作为 FIFO 读时钟，数据必须在同一事务直接返回。
AD1/2 比较器脉冲	被测计数器	控制位先两级同步到各自脉冲域，与 150 MHz 基准计数分开。
FIFO 异步复位	TYFIFO×2	STM32 先禁止读写，再置位 Bit14/15，两路相位模式必须同步复位。

1.6 一次完整测量的事务边界

以 Key4 双路相位测量为例，一次事务不是单个函数，而是横跨 STM32 和 FPGA 的有序协议：

- 1) STM32 通过手动值、比较器或 ADC 自动检测获得信号频率。
- 2) 计算目标采样率和 32 位 NCO 频率字，写入 AD1/AD2 寄存器。
- 3) 置位同步模式，同时复位两个 FIFO，再同时允许写入。

- 4) FPGA 同步装载 AD1=0°、AD2=180° 初相，为同序号样本建立固定配对。
- 5) 两路达到 1024 点完成标志后，STM32 停止写入，依次读出两个 FIFO。
- 6) 验证每个读数的高 4 位为零，完成电压换算、Vpp、基波相位和时差补偿。
- 7) 结果归一化到 (-180°,180°]，再通过 USART 输出。

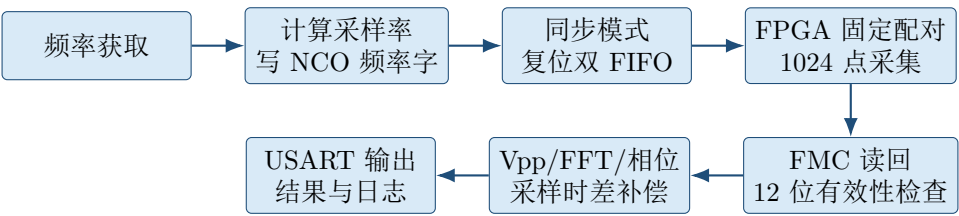


图 2: 双路测量事务的端到端阶段

1.7 数据格式与符号约定

表 3: 跨模块数据格式

数据	格式	解释
AD9226 原始码	12 位偏移二进制	中点约为 2048，STM32 按 $v = 10x/2048 - 10$ 换算到当前 $\pm 10\text{ V}$ 板级量程。
FMC 数据	16 位无符号	ADC 数据低 12 位有效，高 4 位必须为 0；否则该次读回被判定为无效。
NCO 频率字	32 位无符号	高/低 16 位分两个 FMC 地址写入，实际频率受量化影响。
DA 相位	7 位 ROM 偏移	STM32 将 $0 \sim 359^\circ$ 映射到 $0 \sim 127$ 地址，FPGA 与相位累加器高 7 位相加。
FFT 缓冲	1024 点实数	CMSIS-DSP 使用实部/虚部交错复数数组，Hann 窗后进行基 4 FFT。
相位输出	浮点角度	统一定义为 AD2-AD1，并归一化到 $(-180^\circ, 180^\circ]$ 。

2 FPGA 设计

2.1 模块与时钟域

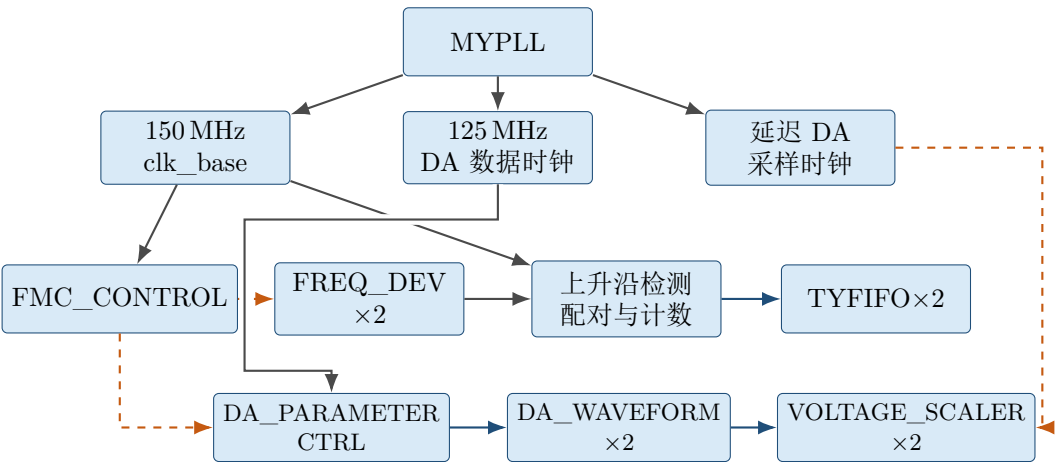


图 3: FPGA 模块连接与主要时钟域

表 4: FPGA 主要模块

模块	职责
TOP.v	顶层连接、采样沿检测、128/1024 点计数和双路配对
FMC_CONTROL.v	地址锁存、16 个写寄存器、16 个读通道和 FIFO 直接读回
FREQ_DEV.v	32 位方波 NCO 与同步初相装载
DA_PARAMETER_CTRL.v	双路 DA 相位累加与 ROM 地址生成
DA_WAVEFORM.v	四种 128 点波形 ROM 选择
VOLTAGE_SCALER_CLOCKED.v	幅度乘法、Q24 缩放、饱和与输出流水
CNT32.v	被测比较器脉冲和 150 MHz 基准计数

2.2 顶层模块的职责与连接策略

TOP.v 是硬件契约的唯一顶层。它不重复实现子模块算法，而是完成五类连接工作：端口引出、时钟分发、FMC 寄存器映射、双路采集配对以及子模块实例化。这种结构的维护原则是：新的数据通道先在独立模块中实现，顶层只增加线网和地址映射。

2.2.1 寄存器线网解包

FMC 输出的 16 个写寄存器在顶层被重命名为有业务意义的线网。例如地址 2/3 对应 DA1 频率字，地址 6/7 对应 AD1 采样 NCO 频率字。这是「总线地址」到「功能信号」的边界，后续模块不再感知 FMC 索引。

```
1 // FMC_CONTROL 已在 clk_base 域保存全部 16 个写寄存器，直接使用即可。
2 assign control_data          = read_data_1;
3 assign da1_frequency_word_high = read_data_2;
4 assign da1_frequency_word_low  = read_data_3;
```

```

5  assign da2_frequency_word_high = read_data_4;
6  assign da2_frequency_word_low  = read_data_5;
7  assign ad1_sample_word_high   = read_data_6;
8  assign ad1_sample_word_low    = read_data_7;
9  assign ad2_sample_word_high   = read_data_8;
10 assign ad2_sample_word_low    = read_data_9;
11 assign da1_wave_select        = read_data_12[7:0];
12 assign da2_wave_select        = read_data_12[15:8];
13 assign da1_amplitude          = read_data_14[11:0];
14 assign da2_amplitude          = read_data_15[11:0];
15
16 // FMC_CONTROL 已完成地址译码，读回数据直接组合映射，避免重复译码产生锁存器。
17 assign ad1_fifo_data_read = {4'b0000,
18     ad1_fifo_data[0], ad1_fifo_data[1], ad1_fifo_data[2], ad1_fifo_data[3],
19     ad1_fifo_data[4], ad1_fifo_data[5], ad1_fifo_data[6], ad1_fifo_data[7],
20     ad1_fifo_data[8], ad1_fifo_data[9], ad1_fifo_data[10], ad1_fifo_data[11]};
21 assign ad2_fifo_data_read = {4'b0000,
22     ad2_fifo_data[0], ad2_fifo_data[1], ad2_fifo_data[2], ad2_fifo_data[3],
23     ad2_fifo_data[4], ad2_fifo_data[5], ad2_fifo_data[6], ad2_fifo_data[7],
24     ad2_fifo_data[8], ad2_fifo_data[9], ad2_fifo_data[10], ad2_fifo_data[11]};
25 assign ad1_fifo_flag_read = (ad1_capture_done || ad1_fifo_full) ? 16'h0001 : 16'h0000;
26 assign ad2_fifo_flag_read = (ad2_capture_done || ad2_fifo_full) ? 16'h0001 : 16'h0000;

```

Listing 1: TOP.v: FMC 寄存器解包与 FIFO 读回格式

代码中的 FIFO 数据并非简单零扩展，而是按当前板级数据线顺序重排 12 个比特。因此 STM32 必须把高 4 位非零视为总线异常，而不能直接使用 `raw\_data & 0x0fff` 掩盖错读。

2.2.2 子模块实例化与双路独立性

AD1 和 AD2 的 NCO、FIFO 均是独立实例，只在同步模式下共享一次初相装载事件。它们不共享 FIFO，也不使用组合逻辑在两个采样时钟之间切换。

```

1  FREQ_DEV ad1_sampling_frequency (
2      .CLK      (clk_base),
3      .EN       (control_data[2]),
4      .PHASE_LOAD(sync_phase_load),
5      .PHASE_INIT(32'h00000000),
6      .FREQH_W  (ad1_sample_word_high),
7      .FREQL_W  (ad1_sample_word_low),
8      .FREQ_OUT(ad1_sampling_clock_nco)
9  );
10
11 FREQ_DEV ad2_sampling_frequency (
12     .CLK      (clk_base),
13     .EN       (control_data[3]),
14     .PHASE_LOAD(sync_phase_load),
15     .PHASE_INIT(32'h80000000),
16     .FREQH_W  (ad2_sample_word_high),
17     .FREQL_W  (ad2_sample_word_low),

```



```

18     .FREQ_OUT(ad2_sampling_clock_nco)
19 );
20
21 TYFIFO ad1_fifo (
22     .data    (AD1_INPUT),
23     .aclr    (control_data[14]),
24     .wrreq   (control_data[4] & ad1_sample_strobe & ~ad1_capture_done),
25     .wrclk   (clk_base),
26     .rdreq   (control_data[5]),
27     .rdclk   (fmc_read_enable),
28     .wrfull  (ad1_fifo_full),
29     .q       (ad1_fifo_data)
30 );
31
32 TYFIFO ad2_fifo (
33     .data    (AD2_INPUT),
34     .aclr    (control_data[15]),
35     .wrreq   (control_data[6] & ad2_capture_strobe & ~ad2_capture_done),
36     .wrclk   (clk_base),
37     .rdreq   (control_data[7]),
38     .rdclk   (fmc_read_enable),
39     .wrfull  (ad2_fifo_full),
40     .q       (ad2_fifo_data)

```

Listing 2: TOP.v: 双路独立 NCO 与独立 FIFO 实例

其中 AD1 初相为 0x00000000, AD2 初相为 0x80000000, 对应 32 位相位空间的半周。双 FIFO 的写时钟都是 `clk_base`, 只有写请求的时刻由各自采样沿决定。

2.3 FMC 地址与直接读回

同一地址的读写方向可以对应不同资源, 见表 5。

表 5: FMC 地址映射

索引	STM32 写入 FPGA	STM32 从 FPGA 读取
0x00	保留	0
0x01	全局控制寄存器	控制寄存器回读
0x02/03	DA1 频率字高/低	AD1 基准计数高/低
0x04/05	DA2 频率字高/低	AD2 基准计数高/低
0x06/07	AD1 采样频率字高/低	AD1 FIFO 数据/完成标志
0x08/09	AD2 采样频率字高/低	AD2 FIFO 数据/完成标志
0x0A/0B	DA1/DA2 相位	AD1 比较器计数高/低
0x0C/0D	波形选择/步进参数	AD2 比较器计数高/低
0x0E/0F	DA1/DA2 幅度缩放值	0

地址 0x06 和 0x08 的 FIFO 数据不进入 150 MHz 读缓存, 而在同一次 FMC 读事务中直接

驱动总线。该路径避免在异步 FMC 读沿和 `clk_base` 之间拼出相邻 FIFO 字的混合码。

2.3.1 FMC 事务译码

STM32F429 在 Bank2 上使用复用地址/数据总线。地址阶段中 `NADV` 为低，FPGA 锁存 `FPGA_DB`；数据阶段再由 `NWE` 或 `NOE` 区分写入与读取。

```

1 // 数据阶段读写使能。
2 assign fmc_wr_en = ((!fpga_cs_ne1) & (!fpga_wr_nwe) & fpga_nl_nadv);
3
4 assign fmc_rd_en = ((!fpga_cs_ne1) & (!fpga_rd_noe) & fpga_nl_nadv);
5
6 // FIFO的q由FMC读沿更新，必须直接在同一读事务中返回；若再由clk域寄存，
7 // 会在两个异步时钟域之间偶发采到相邻FIFO字的混合码。
8 assign fifo_read_address = (addr == 16'h0006) || (addr == 16'h0008);
9 assign fifo_read_data = (addr == 16'h0006) ? write_data_6_ : write_data_8_;
10 assign fpga_db = fmc_rd_en ? (fifo_read_address ? fifo_read_data : rd_data_reg)
11                      : 16'hzzzz;
12
13 // 复用总线地址阶段锁存；该自反馈写法综合为锁存器，板级FMC时序必须满足要求。
14 assign addr = ((fpga_nl_nadv == 1'b0) & (fpga_cs_ne1 == 1'b0)) ? fpga_db : addr;

```

Listing 3: FMC_CONTROL.v: 地址、读写使能与三态总线

- 1) `fmc_wr_en` 表示 STM32 向 FPGA 写，数据被保存到 `read_data*_reg`。
- 2) `fmc_rd_en` 表示 STM32 从 FPGA 读，普通资源先进入 `rd_data_reg`。
- 3) 只有 FPGA 处于读数据阶段时才驱动 `fpga_db`，其余时间输出高阻。
- 4) 地址锁存采用自反馈组合写法，综合后是锁存器，必须依赖板级 FMC 时序满足建立与保持时间。

2.3.2 为什么 FIFO 必须直接读回

TYFIFO 的 `rdclk` 直接连接 `fmc_read_enable`。读事务的边沿推进 FIFO 读指针，而 `q` 的新值属于这一异步事务。如果再用 `clk_base` 寄存该数据，会在两个无固定相位关系的边沿之间采到过渡值。因此地址 `0x06/0x08` 使用专用组合旁路。

```

1 // FIFO的q由FMC读沿更新，必须直接在同一读事务中返回；若再由clk域寄存，
2 // 会在两个异步时钟域之间偶发采到相邻FIFO字的混合码。
3 assign fifo_read_address = (addr == 16'h0006) || (addr == 16'h0008);
4 assign fifo_read_data = (addr == 16'h0006) ? write_data_6_ : write_data_8_;
5 assign fpga_db = fmc_rd_en ? (fifo_read_address ? fifo_read_data : rd_data_reg)
6                      : 16'hzzzz;

```

Listing 4: FMC_CONTROL.v: FIFO 数据同一读事务直通

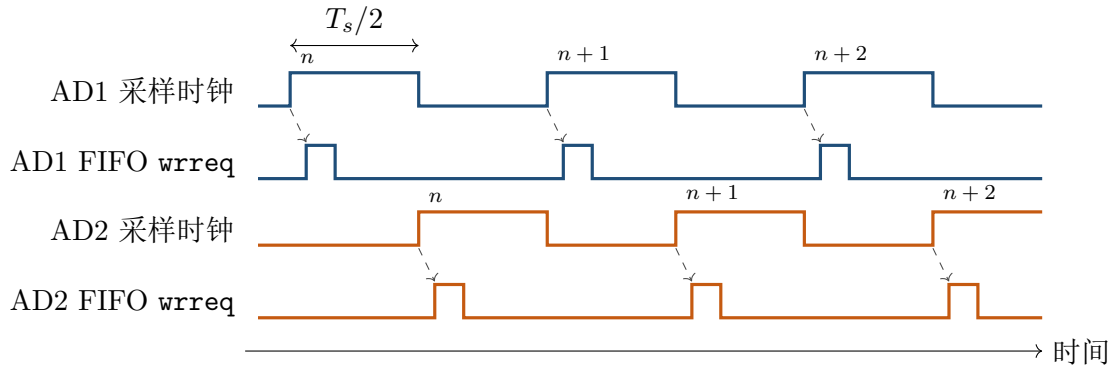
该旁路是当前双 AD 跳码问题修复后的重要契约。后续如果更换 FIFO IP 的 `show-ahead` 模式、修改 FMC 读时序或加入跨时钟域缓存，必须重新验证连续读数是否一字一样本。

2.4 ADC 采集与固定配对

ADC 采样输出始终直连各自 NCO。进入同步模式时，同一 `clk_base` 周期装载：

$$A_1[0] = 0, \quad A_2[0] = 2^{31}. \quad (2.1)$$

两累加器相差半周，因此 AD2 上升沿相对 AD1 上升沿固定延后半周采样周期。两路写请求均在 `clk_base` 域生成；每个采样上升沿被检测后，对应通道在下一个 150 MHz 周期写入各自 FIFO。



虚线：采样上升沿检测后，下一个 $T_{clk} = 1/(150 \text{ MHz})$ 周期产生写请求；横向延时未按比例绘制

图 4: 双路采样沿与各自 FIFO 写入的固定配对

2.4.1 上升沿检测与写请求延迟

NCO 的方波输出同时送往 AD9226 和 FPGA 内部边沿检测逻辑。FPGA 不直接把 NCO 输出当作 FIFO 写时钟，而是在 150 MHz 域中检测上升沿并生成单周期使能。由于非阻塞赋值在时钟边沿后生效，FIFO 在下一个 `clk_base` 边沿观察到写请求，从而给 ADC 并行输出留出稳定时间。

```

1 // ADC采样时钟由NCO输出到AD芯片；上升沿检测后再延迟一个150MHz周期写入FIFO，
2 // 给ADC并行数据留出约13.3ns稳定时间，同时避免使用逻辑生成时钟驱动FIFO。
3 always @(posedge clk_base) begin
4     sync_mode_d <= control_data[1];
5     ad1_sampling_clock_d <= ad1_sampling_clock;
6     ad2_sampling_clock_d <= ad2_sampling_clock;
7     ad1_sample_strobe <= ad1_sampling_clock & ~ad1_sampling_clock_d;
8     ad2_sample_strobe <= ad2_sampling_clock & ~ad2_sampling_clock_d;
9 end
10
11 assign sync_phase_load = control_data[1] & ~sync_mode_d;
12
13 // 同步采集必须先记录AD1上升沿，再接受随后到来的AD2上升沿，确保两个
14 // FIFO的第n个样本始终相差半个采样周期，而不受写使能落在哪个半周期影响。
15 always @(posedge clk_base) begin
16     if (!control_data[1] || !control_data[4] || !control_data[6])
17         sync_capture_started <= 1'b0;
18     else if (ad1_sample_strobe)
19         sync_capture_started <= 1'b1;

```

```

20 end
21
22 assign ad2_capture_strobe = control_data[1]
23 ? (ad2_sample_strobe & sync_capture_started) : ad2_sample_strobe;

```

Listing 5: TOP.v: 采样沿检测、初相装载和 AD2 起始门控

`sync_capture_started` 解决的不是两路频率不一致，而是采集使能可能在任意半周期被 STM32 置位。同步模式下必须先看到 AD1 上升沿，才允许随后的 AD2 上升沿成为第 0 号样本，由此确保两个 FIFO 的第 n 个数据始终属于同一配对。

2.4.2 128/1024 点截止条件

Bit12/13 选择 128 点短采集，用于自动测频的粗测阶段；清零时默认为 1024 点，用于正式 Vpp、FFT 和相位分析。完成标志在最后一个有效写请求后锁存，直到 STM32 清除对应写使能。

```

1  always @(posedge clk_base) begin
2      if (!control_data[4]) begin
3          ad1_capture_count <= 10'd0;
4          ad1_capture_done <= 1'b0;
5      end else if (ad1_sample_strobe && !ad1_capture_done) begin
6          if ((control_data[12] && ad1_capture_count == 10'd127) ||
7              (!control_data[12] && ad1_capture_count == 10'd1023))
8              ad1_capture_done <= 1'b1;
9          else
10             ad1_capture_count <= ad1_capture_count + 1'b1;
11     end
12
13     if (!control_data[6]) begin
14         ad2_capture_count <= 10'd0;
15         ad2_capture_done <= 1'b0;
16     end else if (ad2_capture_strobe && !ad2_capture_done) begin
17         if ((control_data[13] && ad2_capture_count == 10'd127) ||
18             (!control_data[13] && ad2_capture_count == 10'd1023))
19             ad2_capture_done <= 1'b1;
20         else
21             ad2_capture_count <= ad2_capture_count + 1'b1;
22     end
23 end

```

Listing 6: TOP.v: 采集点数计数和完成标志

这段计数逻辑与 FIFO 深度是契约关系：如果改变 `TYFIFO` 深度或 STM32 缓冲区长度，必须同时修改截止值、完成标志、FFT 点数与测试用例，不能只改其中一处。

2.5 采样 NCO 推导

32 位累加器满足

$$A[n+1] = (A[n] + M) \bmod 2^{32}. \quad (2.2)$$

每跨越 2^{32} 完成一个输出周期，故

$$f_s = f_{clk} \frac{M}{2^{32}}, \quad M = \text{round} \left(\frac{f_s}{f_{clk}} 2^{32} \right). \quad (2.3)$$

ADC NCO 使用 $f_{clk} = 150 \text{ MHz}$ 。量化后的实际采样率为

$$f_{s,actual} = 150 \text{ MHz} \frac{M}{2^{32}}. \quad (2.4)$$

2.5.1 NCO 关键代码解析

```

1 module FREQ_DEV (
2     input          CLK,
3     input          EN,
4     input          PHASE_LOAD,
5     input [31:0] PHASE_INIT,
6     input [15:0] FREQH_W,
7     input [15:0] FREQL_W,
8     output reg     FREQ_OUT
9 );
10
11 reg [31:0] FREQ_WORD;
12 reg [31:0] ACC = 32'd0;
13
14 always @(posedge CLK) begin
15     if (PHASE_LOAD) begin
16         ACC <= PHASE_INIT;
17         FREQ_OUT <= PHASE_INIT[31];
18     end else begin
19         if (EN) begin
20             ACC <= ACC + FREQ_WORD;
21         end
22         FREQ_OUT <= ACC[31];
23     end
24 end
25
26 // FMC寄存器在EN关闭时更新；此处再同步到NCO域。
27 always @(posedge CLK) begin
28     FREQ_WORD <= {FREQH_W, FREQL_W};
29 end
30
31 endmodule

```

Listing 7: FREQ_DEV.v: 32 位累加器与可装载初相

代码的优先级是 $\text{PHASE_LOAD} > \text{EN}$ 。进入双路同步模式时，即使 NCO 已经启动，两个实例仍会在同一个 clk_base 边沿分别装载 0° 和 180° 。FREQ_WORD 每个周期都从高/低寄存器重新组合并寄存，STM32 通过先停止相关功能、再分两次写入 16 位字来避免中间值成为有效配置。

量化误差可写为

$$\varepsilon_f = f_{s,actual} - f_{s,qquad} |\varepsilon_f| \leq \frac{f_{clk}}{2^{33}}. \quad (2.5)$$

对 150 MHz 时钟，四舍五入频率字的理论最大量化误差约为 0.0175 Hz。这是数字频率字误差，不等于板级晶振精度或整机测频精度。

2.6 DA 地址与幅度流水

DA 使用固定 125 MHz 时钟。相位累加器高 7 位形成 128 点 ROM 地址：

$$k[n] = \left\lfloor \frac{A[n]}{2^{25}} \right\rfloor, \quad k_{out}[n] = (k[n] + p) \bmod 128. \quad (2.6)$$

其中 p 为相位偏移，等效角度为 $360^\circ p/128$ 。

ROM 中点 $C = 8191$ ，幅度控制值 $a \in [0, 3080]$ ，理想缩放为

$$y = C + (x - C) \frac{a}{3080}. \quad (2.7)$$

硬件使用 Q24 常数 $R = 5447 \approx 2^{24}/3080$ ：

$$|y - C| \approx \left\lfloor \frac{|x - C| a R}{2^{24}} \right\rfloor. \quad (2.8)$$

2.6.1 DA 频率、相位与 ROM 地址

```

1 module DA_PARAMETER_CTRL #(
2     parameter PHASE_WIDTH = 7
3 ) (
4     input                CLK_DA,
5     input                EN,
6     input  [15:0]        FREQA_H_W,
7     input  [15:0]        FREQA_L_W,
8     input  [15:0]        FREQB_H_W,
9     input  [15:0]        FREQB_L_W,
10    input  [15:0]        PHASEA_IN,
11    input  [15:0]        PHASEB_IN,
12    output [PHASE_WIDTH-1:0] COUT_A_FINAL,
13    output [PHASE_WIDTH-1:0] COUT_B_FINAL
14 );
15
16 reg  [31:0] frequency_word_a = 32'd0;
17 reg  [31:0] frequency_word_b = 32'd0;
18 reg  [PHASE_WIDTH-1:0] phase_offset_a = {PHASE_WIDTH{1'b0}};
19 reg  [PHASE_WIDTH-1:0] phase_offset_b = {PHASE_WIDTH{1'b0}};
20 reg  [31:0] accumulator_a = 32'd0;
21 reg  [31:0] accumulator_b = 32'd0;
22
23 always @(posedge CLK_DA) begin
24     if (EN) begin

```

```

25     accumulator_a <= accumulator_a + frequency_word_a;
26     accumulator_b <= accumulator_b + frequency_word_b;
27 end else begin
28     frequency_word_a <= {FREQA_H_W, FREQA_L_W};
29     frequency_word_b <= {FREQB_H_W, FREQB_L_W};
30     phase_offset_a <= PHASEA_IN[PHASE_WIDTH-1:0];
31     phase_offset_b <= PHASEB_IN[PHASE_WIDTH-1:0];
32     accumulator_a <= 32'd0;
33     accumulator_b <= 32'd0;
34 end
35 end
36
37 assign COUT_A_FINAL = accumulator_a[31 -: PHASE_WIDTH] + phase_offset_a;
38 assign COUT_B_FINAL = accumulator_b[31 -: PHASE_WIDTH] + phase_offset_b;
39
40 endmodule

```

Listing 8: DA_PARAMETER_CTRL.v: 双路相位累加器

EN=0 时模块不是仅停止累加,而是同时更新频率字、相位偏移并将累加器清零。因此 STM32 的配置流程必须是「停止 → 写入全部参数 → 启动」。若在 EN=1 期间只改一个 16 位字,硬件可能短暂使用新高字和旧低字组成的频率字。

```

1  always @(*) begin
2      case (WAVE_SEL)
3          SQUARE_WAVE: selected_wave_data = squ_data;
4          TRIANGLE_WAVE: selected_wave_data = tri_data;
5          SAWTOOTH_WAVE: selected_wave_data = swt_data;
6          SINE_WAVE: selected_wave_data = sin_data;
7          default: selected_wave_data = sin_data;
8      endcase
9  end
10
11 always @(posedge CLK) begin
12     wave_data_pipe1 <= selected_wave_data;
13     wave_data_pipe2 <= wave_data_pipe1;
14 end
15
16 assign WAVE_OUT = wave_data_pipe2;

```

Listing 9: DA_WAVEFORM.v: 四种 ROM 选择与两级输出流水

四个同步 ROM 共享同一相位地址,选择器输出再经两级寄存。该延时对两个通道对称,因此不改变相对相位;但如果后续只向其中一路增加流水级,则必须在另一路增加等长延时。

2.6.2 Q24 幅度缩放流水

```

1  localparam [11:0] MAX_VOLTAGE_MV = 12'd3080;
2  localparam [12:0] RECIPROCAL_Q24 = 13'd5447;
3

```

```

4  reg [11:0] voltage_meta = 12'd0;
5  reg [11:0] voltage_sync = 12'd0;
6  reg [13:0] magnitude_pipe1 = 14'd0;
7  reg      positive_pipe1 = 1'b0;
8  reg [24:0] product_pipe2 = 25'd0;
9  reg      positive_pipe2 = 1'b0;
10 reg [37:0] reciprocal_product_pipe3 = 38'd0;
11 reg      positive_pipe3 = 1'b0;
12 reg [13:0] scaled_data_pipe4 = HALF_ROM_MAX;
13
14 wire [13:0] scaled_magnitude = reciprocal_product_pipe3[37:24];
15
16 always @(posedge clk) begin
17     voltage_meta <= (voltage_mv > MAX_VOLTAGE_MV) ? MAX_VOLTAGE_MV : voltage_mv;
18     voltage_sync <= voltage_meta;
19
20     positive_pipe1 <= (rom_data >= HALF_ROM_MAX);
21     magnitude_pipe1 <= (rom_data >= HALF_ROM_MAX)
22         ? (rom_data - HALF_ROM_MAX)
23         : (HALF_ROM_MAX - rom_data);
24
25     product_pipe2 <= magnitude_pipe1 * voltage_sync;
26     positive_pipe2 <= positive_pipe1;
27
28     reciprocal_product_pipe3 <= product_pipe2 * RECIPROCAL_Q24;
29     positive_pipe3 <= positive_pipe2;
30
31     scaled_data_pipe4 <= positive_pipe3
32         ? (HALF_ROM_MAX + scaled_magnitude)
33         : (HALF_ROM_MAX - scaled_magnitude);
34 end
35
36 // 数据在125MHz时钟下降沿更新，外部AD9764在同一时钟上升沿采样。
37 // 分级运算与最终输出错开半个周期，避免数据翻转贴近采样边沿。
38 always @(negedge clk) begin
39     if (!enable)
40         scaled_data <= HALF_ROM_MAX;
41     else
42         scaled_data <= scaled_data_pipe4;
43 end

```

Listing 10: VOLTAGE_SCALER_CLOCKED.v: 限幅、乘法、倒数缩放与半周期输出

流水线将运算拆成四个正沿阶段：幅值/符号分解、乘幅度控制值、乘 Q24 倒数、恢复中点与符号。最终输出在负沿更新，外部 AD9764 在延时后的上升沿采样。voltage_mv 先经两级同步并被限制到 3080，这是硬件安全上限，不应由 STM32 绕过。

2.7 比较器门控计数

每个比较器通道实例化一个 CNT32。模块内包含两个独立时钟域：被测脉冲域累加 N_x ，150 MHz 基准域累加 N_b 。STM32 结束门控后同时读取两者，用计数比消除 HAL_Delay(100) 的实际误差。

```

1 module CNT32 (
2     input        CLR,          // 被测信号计数器同步清零，低有效
3     input        CLRBASE,      // 基准计数器同步清零，低电平有效
4     input        CLK,          // 被测比较器脉冲
5     input        CLKBASE,      // 基准时钟
6     input        CLKEN,        // 被测信号计数使能
7     input        CLKBASEEN,    // 基准计数使能
8     output [31:0] Q,
9     output [31:0] QBASE
10 );
11
12 reg [31:0] Q1 = 32'd0;
13 reg [31:0] Q1BASE = 32'd0;
14
15 always @(posedge CLK) begin
16     if (!CLR) begin
17         Q1 <= 32'b0;
18     end
19     else if (CLKEN) begin
20         Q1 <= Q1 + 1;
21     end
22 end
23
24 assign Q = Q1;
25
26 always @(posedge CLKBASE) begin
27     if (!CLRBASE) begin
28         Q1BASE <= 32'b0;
29     end
30     else if (CLKBASEEN) begin
31         Q1BASE <= Q1BASE + 1;
32     end
33 end
34
35 assign QBASE = Q1BASE;
36
37 endmodule

```

Listing 11: CNT32.v: 被测脉冲与基准时钟的独立计数器

清零信号低有效且是同步清零。比较器域的使能/清零位在 TOP.v 中经两级寄存同步，停止后 STM32 额外等待 1 ms 再读数，避免在跨域控制仍未生效时读取正在变化的计数值。

3 STM32 软件设计

3.1 软件分层

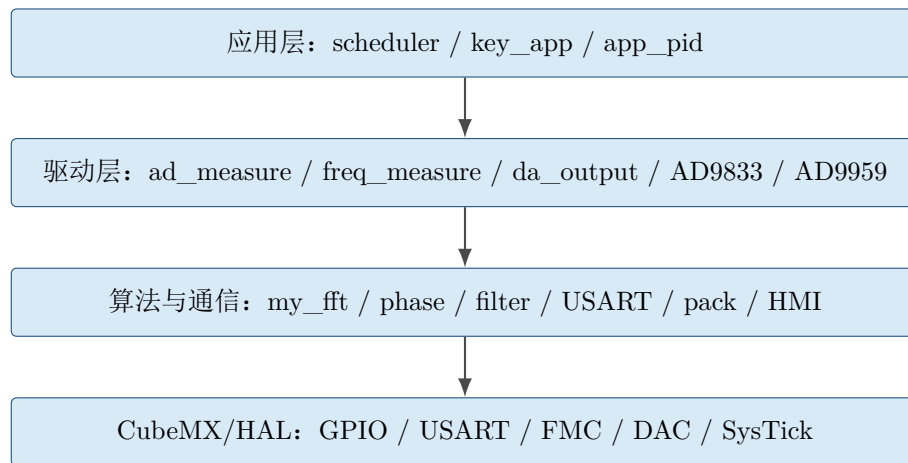


图 5: STM32 软件分层

当前调度表只启用 `key_proc` (10ms) 和 `AD9959_Modulation_State_Update` (5ms)。所有任务在主循环上下文执行，任一阻塞任务都会推迟其他任务。

3.2 启动流程与模块初始化顺序

`main()` 先完成 HAL 和系统时钟，再初始化 GPIO、FMC、USART 和片内 DAC。用户模块必须在 FMC 已可访问后初始化，否则 `CTRL_INIT()` 和 `DA_Apply_Settings()` 会访问尚未建立的外部存储器总线。

```

1 int main(void)
2 {
3     /* USER CODE BEGIN 1 */
4     uint16_t ASK_amp_ch2[16] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
5     /* USER CODE END 1 */
6
7     /* MCU Configuration-----*/
8
9     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
10    HAL_Init();
11
12    /* USER CODE BEGIN Init */
13
14    /* USER CODE END Init */
15
16    /* Configure the system clock */
17    SystemClock_Config();
18
19    /* USER CODE BEGIN SysInit */
20
21    /* USER CODE END SysInit */

```

```

22
23  /* Initialize all configured peripherals */
24  MX_GPIO_Init();
25  MX_FMC_Init();
26  MX_USART1_UART_Init();
27  MX_USART2_UART_Init();
28  MX_USART3_UART_Init();
29  MX_DAC_Init();
30  /* USER CODE BEGIN 2 */
31  HAL_UART_Receive_IT(&huart1, &rxTemp1, 1);
32  HAL_UART_Receive_IT(&huart2, &rxTemp2, 1);
33  HAL_UART_Receive_IT(&huart3, &rxTemp3, 1);
34  CTRL_INIT();
35  DA_Init();
36  DA_Apply_Settings();
37  PID_Init();
38  fft_init();
39
40  /* --- AD9959 初始化与配置流程 --- */
41
42  // 1. 初始化AD9959芯片 (复位, 设置PLL倍频及基础工作模式)
43  AD9959_Init();
44
45  // 2. 初始化通道2的调制功能: 设置为2电平 (LEVEL_MOD_2), ASK模式 (MOD_ASK)
46  // 注意: 新版函数参数顺序已调整, 且无需传入 SWEEP_DISABLE
47  AD9959_Modulation_Init(CH2_SELECT, LEVEL_MOD_2, MOD_ASK);
48
49  // 3. 设置ASK调制的两个幅度值
50  // 当Profile引脚(P2)为低电平时, 输出基准幅度, 代表逻辑'0'
51  ASK_amp_ch2[0] = 0;
52  // 当Profile引脚(P2)为高电平时, 切换到Profile 0, 输出此幅度, 代表逻辑'1'
53  ASK_amp_ch2[1] = 1023; // 使用最大幅度1023以获得最佳信噪比
54
55  // 4. 设置ASK参数: 为通道2配置1000Hz的载波, 并应用上面的幅度设置
56  // 注意: 新版函数参数顺序为 (通道, 频率, 幅度数组)
57  // 该函数会自动将 ASK_amp_ch2[0] 设为基准幅度, ASK_amp_ch2[1] 设为Profile 0的幅度。
58  AD9959_Set_ASK(CH2_SELECT, 1000, ASK_amp_ch2);
59
60  AD9833_Setup(AD9833_REG_FREQ0, 30000.0, AD9833_REG_PHASE1, 1024, AD9833_OUT_TRIANGLE);
61  AD9833_Setup2(AD9833_REG_FREQ0, 30000.0, AD9833_REG_PHASE1, 2048, AD9833_OUT_TRIANGLE);
62  my_printf(&huart1, "ok!\r\n");
63
64  scheduler_init();
65  /* USER CODE END 2 */
66
67  /* Infinite loop */
68  /* USER CODE BEGIN WHILE */
69  while (1)
70  {
71      /* USER CODE END WHILE */

```

```

72
73     /* USER CODE BEGIN 3 */
74
75     scheduler_run();
76 }
77 /* USER CODE END 3 */
78 }
79
80 /**

```

Listing 12: main.c: 外设、FPGA 控制、DDS 和调度器初始化

初始化顺序可归纳为四层：

- 1) HAL/系统时钟：建立 SysTick 和所有 HAL 时序的基准。
- 2) 物理外设：GPIO、FMC、USART、DAC。
- 3) 功能模块：FPGA 控制字、DA 影子配置、PID、FFT 窗函数。
- 4) 外部器件与调度：AD9959、两片 AD9833、静态任务表。

当前 AD9959/AD9833 初始化参数属于演示配置，不是 Key4 测量的前置条件。维护时应把「内部 AD9764 波形路径」与「外部 DDS 路径」视为两个独立功能。

3.3 非抢占式调度器

```

1  * @brief 任务控制块结构体定义
2  * @note 每个任务都由一个该结构体来描述。
3  */
4  typedef struct
5  {
6      void (*task_func)(void);
7      uint32_t rate_ms;
8      uint32_t last_run;
9  } task_t;
10
11 // 兼容原工程保留，当前调度流程未使用。
12 u32 i = 0;
13
14 /**
15  * @brief 输出AD2反馈换算值和PID输出
16  * @param None
17  * @retval None
18  */
19 void uart_proc(void)
20 {
21     my_printf(&huart1, "%f,%d\r\n", vol_amp2 / 2 * 10, output);
22 }
23

```

```

24  /**
25   * @brief 当前启用的静态任务表
26   */
27  static task_t scheduler_task[] =
28  {
29      {key_proc, 10, 0},
30      {AD9959_Modulation_State_Update, 5, 0},
31  };
32
33  /**
34   * @brief 调度器初始化函数
35   * @details 必须在首次调用`scheduler_run()`前执行。
36   * @param None
37   * @retval None
38   */
39  void scheduler_init(void)
40  {
41      task_num = sizeof(scheduler_task) / sizeof(task_t);
42  }
43
44  /**
45   * @brief 调度器主运行函数
46   * @details 在主循环中轮询任务表；无补跑机制，任务执行时间会影响后续任务。
47   * @param None
48   * @retval None
49   */
50  void scheduler_run(void)
51  {
52      for (uint8_t i = 0; i < task_num; i++)
53      {
54          uint32_t now_time = HAL_GetTick();
55
56          // 无符号差值兼容HAL_GetTick回绕。
57          if (now_time - scheduler_task[i].last_run >= scheduler_task[i].rate_ms)
58          {
59              scheduler_task[i].last_run = now_time;
60              scheduler_task[i].task_func();
61          }
62      }

```

Listing 13: scheduler.c: 静态任务表和基于 HAL_GetTick 的轮询

调度器每次进入 `scheduler_run()` 都从头到尾遍历任务表。它不保存「应执行但错过的次数」，也不为任务设置超时。因此 Key4 中的 FIFO 等待、FFT 和大量 USART 输出会阻塞 AD9959 调制状态更新。这是当前架构的已知上限，不应在未修改调度模型时宣称严格 5 ms 调制周期。

3.4 FMC 内存映射与控制位 API

STM32 使用基地址 0x64000000，寄存器索引左移 1 位得到 16 位寄存器地址。同一索引在读、写方向可以映射不同资源，例如索引 6 写入 AD1 采样频率字高半字，读取则推进 AD1 FIFO。

```

1  */
2  enum FPGA_CTRL_BITS
3  {
4      DA_FREQ_EN = (1 << 0), // Bit 0: DA波形输出总使能 (1:启动)
5      AD_SYNC_MODE = (1 << 1), // Bit 1: 同步装载双NCO初相并启用固定样本配对
6      AD1_FREQ_EN = (1 << 2), // Bit 2: AD1采样时钟使能 (1:启动)
7      AD2_FREQ_EN = (1 << 3), // Bit 3: AD2采样时钟使能 (1:启动)
8      AD1_FIFO_WR = (1 << 4), // Bit 4: AD1数据写入FIFO使能 (1:允许写入)
9      AD1_FIFO_RD = (1 << 5), // Bit 5: MCU从AD1 FIFO读取数据使能 (1:允许读取)
10     AD2_FIFO_WR = (1 << 6), // Bit 6: AD2数据写入FIFO使能 (1:允许写入)
11     AD2_FIFO_RD = (1 << 7), // Bit 7: MCU从AD2 FIFO读取数据使能 (1:允许读取)
12     AD1_FREQ_CLR = (1 << 8), // Bit 8: AD1测频计数器清零控制 (0:清除, 1:正常工作, 低电平有效)
13     AD1_FREQ_START = (1 << 9), // Bit 9: AD1测频过程启动控制 (1:启动计数)
14     AD2_FREQ_CLR = (1 << 10), // Bit 10: AD2测频计数器清零控制 (0:清除, 1:正常工作, 低电平有效)
15     AD2_FREQ_START = (1 << 11), // Bit 11: AD2测频过程启动控制 (1:启动计数)
16     AD1_FIFO_SHORT = (1 << 12), // Bit 12: AD1短采集模式 (128点)
17     AD2_FIFO_SHORT = (1 << 13), // Bit 13: AD2短采集模式 (128点)
18     AD1_FIFO_RESET = (1 << 14), // Bit 14: AD1 FIFO异步复位 (高电平有效)
19     AD2_FIFO_RESET = (1 << 15), // Bit 15: AD2 FIFO异步复位 (高电平有效)
20 };
21
22 //=====
23 // 5. FPGA 寄存器地址映射
24 //=====
25 /**
26  * @brief 通过FSMC/FMC将FPGA内部寄存器映射到STM32的内存地址空间。
27  * @details
28  * - 基地址: 0x64000000
29  * - 数据宽度: 16位
30  * - 核心设计: 许多地址在“写”和“读”操作时对应FPGA内部不同的寄存器,
31  * 这是一种节省地址空间的常用设计。
32  */
33 // 基础宏: 计算FPGA寄存器的绝对地址。基地址0x64000000, `addr`是16位寄存器索引。
34 #define reg_addr(addr) ((uint32_t *) (0x64000000 + ((addr) << 1)))
35
36 /** @name 寄存器地址定义 */
37 ///@{
38
39 /** @brief 地址索引 1: 全局控制寄存器 (写入并支持回读) */
40 #define CTRL_DATA *(vu16 *)reg_addr(1)
41
42 /** @brief 地址索引 2, 3: */
43 // 写操作: DA1 输出频率控制字 (32位)
44 #define DA1_H *(vu16 *)reg_addr(2)
45 #define DA1_L *(vu16 *)reg_addr(3)

```

```

46 // 读操作: AD1 频率测量基准时钟(base)的计数值 (32位)
47 #define BASE1_FREQ_H *(vu16 *)reg_addr(2)
48 #define BASE1_FREQ_L *(vu16 *)reg_addr(3)
49
50 /** @brief 地址索引 4, 5: */
51 // 写操作: DA2 输出频率控制字 (32位)
52 #define DA2_H *(vu16 *)reg_addr(4)
53 #define DA2_L *(vu16 *)reg_addr(5)
54 // 读操作: AD2 频率测量基准时钟(base)的计数值 (32位)
55 #define BASE2_FREQ_H *(vu16 *)reg_addr(4)
56 #define BASE2_FREQ_L *(vu16 *)reg_addr(5)
57
58 /** @brief 地址索引 6, 7: */
59 // 写操作: AD1 采样频率控制字 (32位)
60 #define AD1_FS_H *(vu16 *)reg_addr(6)
61 #define AD1_FS_L *(vu16 *)reg_addr(7)
62 // 读操作: AD1 FIFO数据和采集完成标志
63 #define AD1_DATA_SHOW *(vu16 *)reg_addr(6) // 读此地址获取AD1 FIFO中的一个数据
64 #define AD1_FULL_FLAG *(vu16 *)reg_addr(7) // 读此地址获取AD1采集完成标志
65
66 /** @brief 地址索引 8, 9: */
67 // 写操作: AD2 采样频率控制字 (32位)
68 #define AD2_FS_H *(vu16 *)reg_addr(8)
69 #define AD2_FS_L *(vu16 *)reg_addr(9)
70 // 读操作: AD2 FIFO数据和采集完成标志
71 #define AD2_DATA_SHOW *(vu16 *)reg_addr(8) // 读此地址获取AD2 FIFO中的一个数据
72 #define AD2_FULL_FLAG *(vu16 *)reg_addr(9) // 读此地址获取AD2采集完成标志

```

Listing 14: commond_init.h: 全局控制位与主要 FMC 寄存器

cmd_to_fun.c 将位操作封装为启停、复位和 FIFO 读写 API。这些函数都是对 CTRL_DATA 的读-改-写, 所以只能在同一软件上下文中串行使用; 若后续在中断和主循环同时修改该寄存器, 必须增加临界区。

```

1 void AD_FREQ_SET(int ch)
2 {
3     switch (ch) {
4         case 1:
5             CTRL_DATA = CTRL_DATA | AD1_FREQ_EN; // 使能AD1采样时钟
6             break;
7         case 2:
8             CTRL_DATA = CTRL_DATA | AD2_FREQ_EN; // 使能AD2采样时钟
9             break;
10        case 3:
11            CTRL_DATA = CTRL_DATA | AD1_FREQ_EN | AD2_FREQ_EN; // 同时使能两个通道
12            break;
13    }
14 }
15
16 /**
17  * @brief 使能AD数据写入FIFO

```

```

18  * @details 通过置位 `ADx_FIFO_WR` 控制位, 允许AD转换后的数据写入到FPGA内部的FIFO缓存中。
19  * @param ch 通道号 (1: AD1, 2: AD2, 3: AD1和AD2同时)
20  */
21 void AD_FIFO_WRITE_ENABLE(int ch)
22 {
23     switch (ch) {
24     case 1:
25         CTRL_DATA = CTRL_DATA | AD1_FIFO_WR; // 允许AD1数据写入FIFO
26         break;
27     case 2:
28         CTRL_DATA = CTRL_DATA | AD2_FIFO_WR; // 允许AD2数据写入FIFO
29         break;
30     case 3:
31         CTRL_DATA = CTRL_DATA | AD1_FIFO_WR | AD2_FIFO_WR; // 同时允许两个通道写入
32         break;
33     }
34 }
35
36 /**
37  * @brief 禁能AD数据写入FIFO
38  * @details 通过清零 `ADx_FIFO_WR` 控制位, 停止AD数据向FIFO缓存的写入。
39  * @param ch 通道号 (1: AD1, 2: AD2)
40  */
41 void AD_FIFO_WRITE_DISABLE(int ch)
42 {
43     switch (ch) {
44     case 1:
45         CTRL_DATA = CTRL_DATA & (~AD1_FIFO_WR); // 禁止AD1数据写入FIFO
46         break;
47     case 2:
48         CTRL_DATA = CTRL_DATA & (~AD2_FIFO_WR); // 禁止AD2数据写入FIFO

```

Listing 15: cmd_to_fun.c: 采样时钟与 FIFO 写使能的位操作封装

3.5 Key4 测量流程

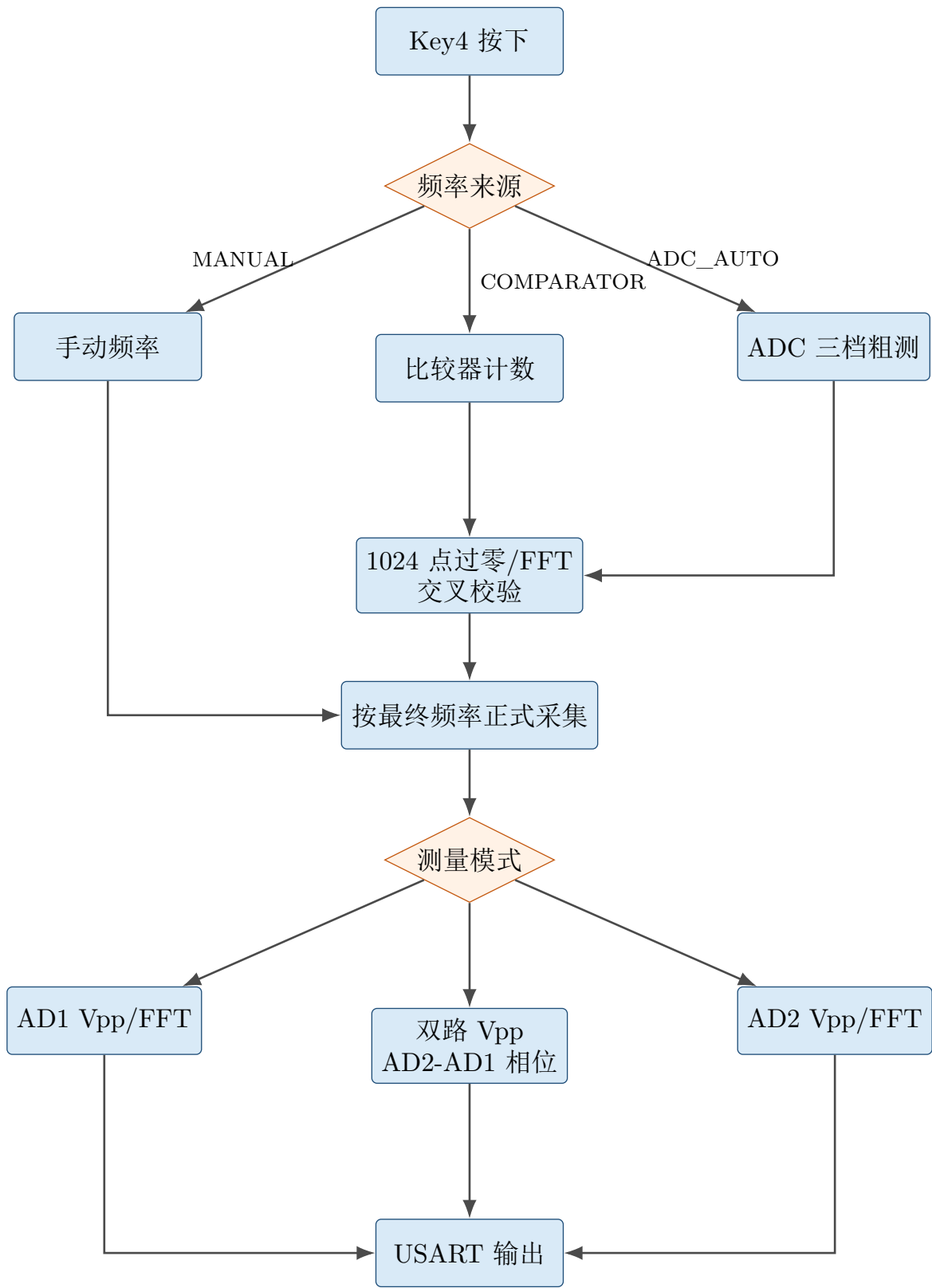


图 6: Key4 频率来源、测量模式和输出流程

3.5.1 按键边沿检测与回调表

```

1 typedef struct {
2     GPIO_TypeDef* port;      // 按键连接的GPIO端口
3     uint16_t      pin;       // 按键连接的GPIO引脚
4     key_callback_t callback; // 按键按下时要调用的回调函数
5 } key_info_t;
6
7 static void key1_action_toggle_ask(void);
8 static void key2_action_toggle_fsk(void);
9 static void key3_action_cycle_phase(void);
10 static void key4_action_perform_fft(void);
11
12 static const key_info_t key_config[NUM_KEYS] = {
13     {KEY1_GPIO_Port, KEY1_Pin, key1_action_toggle_ask}, // 按键1 -> 切换ASK
14     {KEY2_GPIO_Port, KEY2_Pin, key2_action_toggle_fsk}, // 按键2 -> 切换FSK
15     {KEY3_GPIO_Port, KEY3_Pin, key3_action_cycle_phase}, // 按键3 -> 调整相位
16     {KEY4_GPIO_Port, KEY4_Pin, key4_action_perform_fft} // 按键4 -> 测频并采集
17 };
18
19 // 每一位对应key_config中的一个按键。
20 uint8_t key_map_now = 0;
21 uint8_t key_map_old = 0;
22 uint8_t key_map_down = 0;
23
24 /**
25  * @brief 读取所有物理按键的当前状态，并返回一个位掩码(bitmask)
26  * @details 遍历`key_config`数组，扫描每个已定义的按键。如果按键被按下（低电平），
27  * 就在返回值的对应位上置1。例如，按键0按下，返回 0b0001；按键1按下，返回 0b0010。
28  * @return uint8_t 包含所有按键状态的位掩码。
29  */
30 static uint8_t key_read_map(void)
31 {
32     uint8_t map = 0;
33     for (int i = 0; i < NUM_KEYS; i++) {
34         if (HAL_GPIO_ReadPin(key_config[i].port, key_config[i].pin) == GPIO_PIN_RESET) {
35             map |= (1 << i);
36         }
37     }
38     return map;
39 }
40
41 /**
42  * @brief 检测新的按下事件并调用对应回调
43  * @details
44  * 1. 调用`key_read_map()`获取所有按键状态的位掩码。
45  * 2. 计算哪些按键刚刚被按下。
46  * 3. 遍历所有按键，如果检测到某个按键被按下，就调用其在`key_config`数组中注册的回调函数。
47  */
48 void key_proc(void)
49 {
50     key_map_now = key_read_map();

```

```

51  key_map_down = key_map_now & (key_map_old ^ key_map_now); // 逻辑按下边沿
52  // key_map_up = ~key_map_now & (key_map_old ^ key_map_now);
53  key_map_old = key_map_now;
54
55  if (key_map_down == 0) {
56      return;
57  }
58
59  for (int i = 0; i < NUM_KEYS; i++) {
60      if (key_map_down & (1 << i)) {
61          if (key_config[i].callback != NULL) {
62              key_config[i].callback();
63          }
64      }
65  }
66 }

```

Listing 16: key_app.c: 按键配置表、位图扫描和下沿回调

按键低电平表示按下。按下边沿位图计算为

$$K_{down} = K_{now} \wedge (K_{old} \oplus K_{now}), \quad (3.1)$$

因此只在稳态从 0 变为 1 时置位。当前没有独立消抖计时器，而是依赖 10 ms 调度周期和物理按键特性。若实机出现一次按下触发多次，应在这一公共边缘检测层增加消抖，而不是分别修改四个按键回调。

3.5.2 频率来源选择与交叉校验

```

1  static void key4_capture_single(int channel, float signal_freq)
2  {
3      if (channel == 1)
4          vpp_adc_parallel(signal_freq, FREQ_MODE_SIGNAL,
5                          0.0f, FREQ_MODE_SIGNAL);
6      else
7          vpp_adc_parallel(0.0f, FREQ_MODE_SIGNAL,
8                          signal_freq, FREQ_MODE_SIGNAL);
9  }
10
11 static int key4_get_signal_frequency(int channel, float *signal_freq)
12 {
13     #if KEY4_FREQ_SOURCE == KEY4_FREQ_SOURCE_MANUAL
14         *signal_freq = KEY4_MANUAL_SIGNAL_FREQ;
15     #elif KEY4_FREQ_SOURCE == KEY4_FREQ_SOURCE_COMPARATOR
16         if (channel == 1) {
17             freq1 = 0.0f;
18             fre_measure_ad1();
19             *signal_freq = freq1;
20         } else {

```

```

21     freq2 = 0.0f;
22     fre_measure_ad2();
23     *signal_freq = freq2;
24 }
25 if (*signal_freq <= 0.0f) {
26     my_printf(&huart1,
27             "AD%d comparator frequency is invalid; capture cancelled.\r\n",
28             channel);
29     return 0;
30 }
31 #elif KEY4_FREQ_SOURCE == KEY4_FREQ_SOURCE_ADC_AUTO
32 if (!ad_detect_frequency(channel, signal_freq)) {
33     my_printf(&huart1,
34             "AD%d auto frequency detection failed; capture cancelled.\r\n",
35             channel);
36     return 0;
37 }
38
39 // 128点只负责选出安全采样档；正式1024点用过零与Hann FFT交叉校验。
40 key4_capture_single(channel, *signal_freq);
41 float zero_crossing_freq = ad_estimate_captured_frequency(channel);
42 float *samples = (channel == 1) ? fifo_data1_f : fifo_data2_f;
43 calculate_fft_spectrum(samples, AD_FIFO_SIZE);
44 float fft_peak_freq = get_precise_peak_frequency(
45     get_adc_fft_sampling_frequency(channel));
46 float frequency_difference = zero_crossing_freq - fft_peak_freq;
47 if (frequency_difference < 0.0f) frequency_difference = -frequency_difference;
48
49 if (fft_peak_freq > 0.0f &&
50     (zero_crossing_freq <= 0.0f ||
51     frequency_difference > fft_peak_freq * KEY4_FREQ_AGREEMENT_LIMIT))
52     *signal_freq = fft_peak_freq;
53 else
54     *signal_freq = zero_crossing_freq;
55
56 if (*signal_freq <= 0.0f) {
57     my_printf(&huart1,
58             "AD%d frequency refinement failed; capture cancelled.\r\n",
59             channel);
60     return 0;
61 }
62 #else
63 #error "KEY4_FREQ_SOURCE配置无效"
64 #endif
65 return 1;
66 }

```

Listing 17: key_app.c: Key4 频率来源和 ADC/FFT 交叉校验

三种来源由编译期宏选择：手动值用于已知信号，比较器适合幅度充足的整形脉冲，ADC 自动模式则先用 128 点选档，再用 1024 点过零和 Hann FFT 交叉校验。当两种精测结果相差

超过 FFT 结果的 1% 时，优先采用 FFT 峰值频率。

3.5.3 测量模式分流

```

1 static void key4_action_perform_fft(void)
2 {
3     float signal_freq = KEY4_MANUAL_SIGNAL_FREQ;
4     int channel = (KEY4_MEASURE_MODE == KEY4_MODE_AD2_ONLY) ? 2 : 1;
5
6     if (KEY4_MEASURE_MODE == KEY4_MODE_PHASE_PAIR) {
7         if (!key4_get_signal_frequency(1, &signal_freq)) return;
8         my_printf(&huart1, "AD1 Signal Freq: %.3f Hz\r\n", signal_freq);
9         float phase_difference = 0.0f;
10        int phase_valid = ad_measure_phase_difference(
11            signal_freq, KEY4_PHASE_CALIBRATION_NS, &phase_difference);
12        my_printf(&huart1, "AD1 Vpp: %.4f V\r\n", vol_amp1);
13        my_printf(&huart1, "AD2 Vpp: %.4f V\r\n", vol_amp2);
14        if (phase_valid)
15            my_printf(&huart1, "Phase Diff (AD2-AD1): %.3f deg\r\n",
16                phase_difference);
17        else
18            my_printf(&huart1, "Phase Diff (AD2-AD1): invalid\r\n");
19    } else {
20        if (!key4_get_signal_frequency(channel, &signal_freq)) return;
21        my_printf(&huart1, "AD%d Signal Freq: %.3f Hz\r\n",
22            channel, signal_freq);
23        key4_capture_single(channel, signal_freq);
24        my_printf(&huart1, "AD%d Vpp: %.4f V\r\n", channel,
25            (channel == 1) ? vol_amp1 : vol_amp2);
26    }
27
28    #if KEY4_RUN_FFT
29        my_printf(&huart1, "Calculating FFT spectrum...\r\n");
30        fft_analyze_ad_buffer(channel);
31    #endif
32 }

```

Listing 18: key_app.c: 单通道与双路相位模式的最终分流

相位模式始终以 AD1 获得信号频率，然后同时采集两路。单 AD2 模式则会把 FFT 数据指针切换到 fifo_data2_f。这一分流是输出标签、FFT 数据源和物理采集通道保持一致的关键。

3.6 自适应采样率

当信号频率为 f_0 时，优先目标采样率为

$$f_{s,target} = \begin{cases} 128f_0, & f_0 \leq 100 \text{ kHz}, \\ 64f_0, & f_0 > 100 \text{ kHz}. \end{cases} \quad (3.2)$$

若超过 65 MHz，令 1024 点覆盖整数周期数

$$C = \left\lceil \frac{1024f_0}{65 \text{ MHz}} \right\rceil, \quad f_{s,target} = \frac{1024f_0}{C}. \quad (3.3)$$

ADC 自动测频先在 65 MHz、65 MHz/128、65 MHz/128² 三档采集 128 点。幅度接近时优先采用更高采样率候选，随后使用短采集过零精测，并在正式 1024 点采集中与 Hann FFT 结果比较。

3.6.1 正式采样率的代码实现

```

1 static void setSamplingFrequency(float freq, int channel, AdcFrequencyMode mode)
2 {
3     if (freq <= 0.0f || (channel != 1 && channel != 2))
4         return;
5
6     unsigned int fs = 0; // 最终要设置的采样频率 (Sampling Frequency)
7
8     if (mode == FREQ_MODE_SAMPLING)
9     {
10         // 直接指定采样频率模式：直接使用传入的值作为采样率
11         fs = (unsigned int)freq;
12     }
13     else
14     {
15         // 正式幅度/THD统一直接采样；达到上限时改选1024点整数周期采样率。
16         float ratio = (freq <= ADC_DIRECT_RATIO_SWITCH)
17             ? ADC_LOW_DIRECT_RATIO : ADC_HIGH_DIRECT_RATIO;
18         float target_sampling_freq = freq * ratio;
19         if (target_sampling_freq > ADC_MAX_SAMPLE_RATE)
20         {
21             float cycles_exact = freq * AD_FIFO_SIZE / ADC_MAX_SAMPLE_RATE;
22             unsigned int cycles = (unsigned int)cycles_exact;
23             if ((float)cycles < cycles_exact) cycles++;
24             target_sampling_freq = freq * AD_FIFO_SIZE / cycles;
25         }
26         fs = (unsigned int)(target_sampling_freq + 0.5f);
27     }
28
29     if (fs > ADC_MAX_SAMPLE_RATE)
30     {
31         fs = ADC_MAX_SAMPLE_RATE;
32     }
33
34     // 32位NCO频率字：M = fs * 2^32 / 150MHz。
35     unsigned int M = AD_FREQ_CONSTANT * fs / FPGA_BASE_CLK;
36
37     // 通过FMC写入目标采样NCO。
38     if (channel == 1)
39     {

```

```

40     AD_FREQ_SET(1);
41     HAL_Delay(1);
42     AD1_FS_H = M >> 16; // 写入频率控制字的高16位
43     HAL_Delay(1);
44     AD1_FS_L = M & 0xFFFF; // 写入频率控制字的低16位
45 }
46 else
47 {
48     AD_FREQ_SET(2);
49     HAL_Delay(1);
50     AD2_FS_H = M >> 16; // 写入频率控制字的高16位
51     HAL_Delay(1);
52     AD2_FS_L = M & 0xFFFF; // 写入频率控制字的低16位
53 }
54
55 adc_sampling_freq[channel - 1] = (float)fs;
56 adc_fft_sampling_freq[channel - 1] = (float)fs;
57 }
58
59 /**
60  * @brief 用带迟滞的上升沿过零点估算频率。
61  * @note 最少需要4个同向过零点，即覆盖3个完整周期。

```

Listing 19: ad_measure.c: 直接采样比例、65 MHz 上限与 NCO 频率字

当 $128f_0$ 或 $64f_0$ 超过上限时，代码选择一个使 1024 点尽可能覆盖整数周期的采样率。这样做不是为了提高奈奎斯特上限，而是减少窗函数前的截断不连续，同时保留足够的每周期样本数。代码保存的 `adc_sampling_freq` 是根据整数 `fs` 计算的物理采样率，相位补偿必须使用它，不能使用未量化的目标值。

3.6.2 三档粗测与过零精测

```

1 int ad_detect_frequency(int channel, float *frequency_hz)
2 {
3     static const float coarse_sample_rates[] = {
4         65000000.0f,
5         65000000.0f / AD_SHORT_SIZE,
6         65000000.0f / AD_SHORT_SIZE / AD_SHORT_SIZE
7     };
8     float candidate_frequency[3];
9     float candidate_magnitude[3];
10    const float *samples;
11
12    if ((channel != 1 && channel != 2) || frequency_hz == NULL)
13    {
14        return 0;
15    }
16
17    samples = (channel == 1) ? fifo_data1_f : fifo_data2_f;
18    for (int i = 0; i < 3; i++)

```

```

19  {
20      if (!captureAdSamples(channel, coarse_sample_rates[i], AD_SHORT_SIZE) ||
21          !fft_find_dominant_frequency(samples, coarse_sample_rates[i],
22                                       &candidate_frequency[i],
23                                       &candidate_magnitude[i]))
24      {
25          return failFrequencyDetection(channel, frequency_hz);
26      }
27  }
28
29  float max_magnitude = candidate_magnitude[0];
30  for (int i = 1; i < 3; i++)
31      if (candidate_magnitude[i] > max_magnitude)
32          max_magnitude = candidate_magnitude[i];
33
34  if (max_magnitude < ADC_FREQ_MIN_PEAK_VOLTS)
35  {
36      return failFrequencyDetection(channel, frequency_hz);
37  }
38
39  // 多档幅度接近时优先高采样率，避免选择低采样率产生的混叠频率。
40  float coarse_frequency = candidate_frequency[2];
41  for (int i = 0; i < 3; i++)
42  {
43      if (candidate_magnitude[i] >= max_magnitude * ADC_FREQ_PEAK_RATIO)
44      {
45          coarse_frequency = candidate_frequency[i];
46          break;
47      }
48  }
49
50  float fine_ratio = (coarse_frequency > 60000.0f)
51      ? ADC_FREQ_FINE_HIGH_RATIO : ADC_FREQ_FINE_LOW_RATIO;
52  float precise_sample_rate = coarse_frequency * fine_ratio;
53  if (precise_sample_rate > ADC_MAX_SAMPLE_RATE)
54      precise_sample_rate = ADC_MAX_SAMPLE_RATE;
55
56  if (!captureAdSamples(channel, precise_sample_rate, AD_SHORT_SIZE))
57  {
58      return failFrequencyDetection(channel, frequency_hz);
59  }
60
61  const u16 *raw_samples = (channel == 1) ? fifo_data1 : fifo_data2;
62  *frequency_hz = estimateFrequency(raw_samples, AD_SHORT_SIZE,
63                                   get_adc_sampling_frequency(channel));
64  if (*frequency_hz <= 0.0f)
65  {
66      precise_sample_rate *= 0.5f;
67      if (!captureAdSamples(channel, precise_sample_rate, AD_SHORT_SIZE))
68          return failFrequencyDetection(channel, frequency_hz);

```



```

69     *frequency_hz = estimateFrequency(raw_samples, AD_SHORT_SIZE,
70                                     get_adc_sampling_frequency(channel));
71 }
72
73 return (*frequency_hz > 0.0f)
74       ? 1 : failFrequencyDetection(channel, frequency_hz);
75 }

```

Listing 20: ad_measure.c: 多采样档幅度判决和二次过零精测

三个候选频率不直接按频率大小投票，而是先比较 Hann FFT 峰值幅度。当多档幅度都达到最大幅度的 80% 时，从高采样率向低采样率选择第一个合格档，以减少混叠假峰被选中的概率。若第一次过零精测失败，采样率减半后只重试一次；再失败则将频率、采样率和幅度结果全部置为无效。

3.7 比较器门控测频

设同一门控窗口内被测脉冲计数为 N_x ，150 MHz 基准计数为 N_b ，则

$$T_g = \frac{N_b}{150 \text{ MHz}}, \quad f_x = 150 \text{ MHz} \frac{N_x}{N_b}. \quad (3.4)$$

因此计算使用实际门控宽度，而不要求软件延时精确等于标称 100 ms。

```

1 void fre_measure(int channel, u32 *freq_ad, u32 *freq_base, float *freq_result)
2 {
3     // 步骤1: 清除上一次的采集数据，确保本次测量从0开始
4     AD_FREQ_CLR_ENABLE(channel);
5     HAL_Delay(1); // 短暂延时确保FPGA命令执行
6     AD_FREQ_CLR_DISABLE(channel);
7     HAL_Delay(1);
8
9     // 步骤2: 启动FPGA进行频率采集
10    AD_FREQ_START(channel);
11    HAL_Delay(100); // 核心步骤: 设置100ms的门控时间 (Gate Time) 进行计数
12    AD_FREQ_STOP(channel);
13    HAL_Delay(1); // 等待停止命令跨入比较器输入时钟域，确保32位计数值稳定
14
15    // 步骤3: 读取被测信号的计数值
16    // 通过三元运算符根据通道号选择对应的FPGA寄存器
17    u32 freq_h = (channel == 1) ? AD1_FREQ_H : AD2_FREQ_H;
18    HAL_Delay(1);
19    u32 freq_l = (channel == 1) ? AD1_FREQ_L : AD2_FREQ_L;
20    HAL_Delay(1);
21    // 将读取到的高16位和低16位数据合并成一个32位整数
22    *freq_ad = (freq_h << 16) + freq_l;
23
24    // 步骤4: 读取基准时钟的计数值
25    u32 base_h = (channel == 1) ? BASE1_FREQ_H : BASE2_FREQ_H;
26    HAL_Delay(1);
27    u32 base_l = (channel == 1) ? BASE1_FREQ_L : BASE2_FREQ_L;

```

```

28     HAL_Delay(1);
29     // 合并高低16位数据
30     *freq_base = (base_h << 16) + base_l;
31
32     // 步骤5: 数据有效性检查
33     // 无被测脉冲或100ms门控内基准计数低于14,000,000时判为无效。
34     if (*freq_ad == 0 || *freq_base <= 14000000.0)
35     {
36         return;
37     }
38
39     // 步骤6: 计算实际频率
40     // 公式:  $F_{actual} = F_{base} * (N_{ad} / N_{base})$ 
41     //  $F_{base} = 150,000,000$  Hz
42     //  $N_{ad} = *freq\_ad$  (被测信号计数值)
43     //  $N_{base} = *freq\_base$  (基准时钟计数值)
44     *freq_result = FPGA_BASE_CLK / *freq_base * *freq_ad;
45 }
46
47 /**
48  * @brief 测量 AD1 通道的频率
49  * @details

```

Listing 21: freq_measure.c: 清零、门控、读取与计数比测频

读取顺序是先停止、等待跨域控制稳定,再分别读高/低 16 位。当基准计数不足 14,000,000 或被测计数为 0 时,本次测量保持旧结果。调用者在开始新测量前应先将结果变量清零,Key4 比较器路径已按该契约处理。

3.8 DA 输出的软件影子配置

STM32 不在每个 Set 函数中立即写硬件,而是先更新 da_channels[2] 影子配置。DA_Apply_Settings() 把两路配置作为一个事务提交:先停止 FPGA DDS,写入频率、幅度、相位与波形,再重新启动。

```

1 void DA_SetConfig(uint8_t channel_index, float freq, uint16_t amp, uint16_t angle, Waveform_t
   wave)
2 {
3     // 委托给各个独立的Set函数来完成配置更新
4     DA_SetFREQ(channel_index, freq);
5     DA_SetAmp(channel_index, amp);
6     DA_SetPhase(channel_index, angle);
7     DA_SetWaveform(channel_index, wave);
8 }
9
10 /**
11  * @brief 将内存中的配置应用到硬件
12  * @details
13  * 此函数是连接软件配置与硬件行为的核心。它负责:
14  * 1. 请求FPGA暂停波形生成,以确保寄存器更新的原子性和安全性。

```

```

15  * 2. 从 `da_channels` 数组中读取两个通道的完整配置。
16  * 3. 执行必要的数学计算，如频率控制字和相位映射。
17  * 4. 将计算结果写入FPGA内对应的硬件寄存器。
18  * 5. 将波形类型参数打包写入组合寄存器。
19  * 6. 请求FPGA依据新配置重新生成波形。
20  * @note 所有通过Set/SetConfig的修改，必须调用此函数后才能在硬件上生效。
21  */
22 void DA_Apply_Settings(void)
23 {
24     // 1. 请求FPGA暂停输出，以安全更新寄存器
25     DA_FPGA_STOP();
26
27     // 2. --- 通道0 (DA1) 参数计算与写入 ---
28     // 根据NCO(数控振荡器)原理计算32位频率控制字M
29     unsigned int M_DA1 = DA_FREQ_CONSTANT * da_channels[0].frequency / DA_SAMPLE_CLK;
30     // 将32位的M值拆分为高16位和低16位写入硬件寄存器
31     DA1_H = M_DA1 >> 16;
32     DA1_L = M_DA1 & 0x0000FFFF;
33     // 写入幅度寄存器
34     DA1_AMP = da_channels[0].amplitude;
35     // 将0-359度的相位归一化，并映射到128点ROM地址
36     uint16_t normalized_angle1 = da_channels[0].phase % 360;
37     DA1_PHASE = (uint16_t)(roundf(normalized_angle1 * (DA_ROM_SIZE / 360.0f)));
38
39     // 3. --- 通道1 (DA2) 参数计算与写入 ---
40     unsigned int M_DA2 = DA_FREQ_CONSTANT * da_channels[1].frequency / DA_SAMPLE_CLK;
41     DA2_H = M_DA2 >> 16;
42     DA2_L = M_DA2 & 0x0000FFFF;
43     DA2_AMP = da_channels[1].amplitude;
44     uint16_t normalized_angle2 = da_channels[1].phase % 360;
45     DA2_PHASE = (uint16_t)(roundf(normalized_angle2 * (DA_ROM_SIZE / 360.0f)));
46
47     // 4. --- 组合寄存器写入 ---
48     // 将两个通道的波形类型打包到一个16位寄存器中：高8位为通道1，低8位为通道0
49     // 直接赋值可避免读-改-写操作的潜在风险
50     DA_WAVEFORM = (da_channels[1].waveform << 8) | (da_channels[0].waveform);
51
52     // 5. 请求FPGA依据新配置重新生成波形
53     DA_FPGA_START();
54 }
55
56 // =====
57 // [DEPRECATED] 废弃的测试功能
58 // =====
59 // 以下代码为早期开发阶段的测试功能，现已废弃，不应在正式代码中调用。
60 // =====
61
62 /**
63  * @brief 用于非阻塞延时的计时器变量（配合已废弃的wave_test）
64  */

```

```
65 static uint32_t wave_tick = 0;
```

Listing 22: da_output.c: 双通道影子配置与原子提交

频率字按 125 MHz 基准计算；相位先对 360° 取模，再映射到 128 点 ROM 地址。波形寄存器低 8 位为 DA1，高 8 位为 DA2，直接一次赋值可避免对只写寄存器进行无意义的读-改-写。

4 数字信号处理算法

4.1 ADC 电压与峰峰值

12 位偏移二进制码 $x \in [0, 4095]$ 按当前 $\pm 10\text{ V}$ 板级量程换算：

$$v(x) = 10 \frac{x}{2048} - 10. \quad (4.1)$$

整轮原始极值给出

$$V_{pp} = 10 \frac{x_{max} - x_{min}}{2048}. \quad (4.2)$$

该定义对单点毛刺敏感，因此需要结合 FFT 基波幅值判断偶发异常。

4.1.1 FIFO 读回和有效性防线

```
1 static int readFIFOData(int channel, u16 *fifo_data, float *fifo_data_f, int size)
2 {
3     int valid = 1;
4
5     // 读使能同时推进FIFO读指针。
6     if (channel == 1)
7         AD_FIFO_READ_ENABLE(1);
8     else
9         AD_FIFO_READ_ENABLE(2);
10
11     HAL_Delay(1);
12
13     for (int i = 0; i < size; i++)
14     {
15         u16 raw_data = (channel == 1) ? AD1_DATA_SHOW : AD2_DATA_SHOW;
16
17         // FPGA有效ADC数据固定为12位；高4位非零表示本次FMC读回无效。
18         if ((raw_data & 0xF000U) != 0U)
19         {
20             valid = 0;
21             raw_data = (i > 0) ? fifo_data[i - 1] : 0x0800U;
22         }
23         fifo_data[i] = raw_data;
24
25         // 12位偏移二进制码映射到板级双极性电压。
26         fifo_data_f[i] = fifo_data[i] * VOLTAGE_OFFSET / ADC_SCALE - VOLTAGE_OFFSET;
```

```

27     }
28
29     // 关闭读使能，避免后续FMC访问继续推进FIFO。
30     if (channel == 1)
31         AD_FIFO_READ_DISABLE(1);
32     else
33         AD_FIFO_READ_DISABLE(2);
34
35     return valid;
36 }

```

Listing 23: ad_measure.c: FIFO 读指针推进、12 位校验与电压换算

FMC 每读一次数据地址就推进 FIFO 指针，因此读循环中不能额外做“试读”。当高 4 位非零时，代码用上一样本填补缓冲区以避免未初始化数据进入后续算法，但同时把整次采集标记为无效。调用者不会更新 Vpp 或相位结果，因此填补值不会被当作真实测量。

```

1 void vpp_adc_parallel(float freq1, AdcFrequencyMode mode1, float freq2, AdcFrequencyMode mode2)
2 {
3     if (freq1 > 0)
4     {
5         setShortCaptureMode(1, 0);
6         setSamplingFrequency(freq1, 1, mode1);
7         resetFifo(1);
8         AD_FIFO_WRITE_ENABLE(1);
9     }
10
11     if (freq2 > 0)
12     {
13         setShortCaptureMode(2, 0);
14         setSamplingFrequency(freq2, 2, mode2);
15         resetFifo(2);
16         AD_FIFO_WRITE_ENABLE(2);
17     }
18
19     HAL_Delay(1);
20
21     if (freq1 > 0)
22     {
23         while (AD1_FULL_FLAG != 0x0001)
24             ;
25         AD_FIFO_WRITE_DISABLE(1);
26     }
27
28     if (freq2 > 0)
29     {
30         while (AD2_FULL_FLAG != 0x0001)
31             ;
32         AD_FIFO_WRITE_DISABLE(2);
33     }
34

```

```

35     if (freq1 > 0)
36     {
37         if (readFIFOData(1, fifo_data1, fifo_data1_f, AD_FIFO_SIZE))
38         {
39             findMinMax(fifo_data1, AD_FIFO_SIZE, &vol_maxnum1, &vol_minnum1);
40             vol_amp1 = (vol_maxnum1 - vol_minnum1) * VOLTAGE_OFFSET / ADC_SCALE;
41         }
42     }
43
44     if (freq2 > 0)
45     {
46         if (readFIFOData(2, fifo_data2, fifo_data2_f, AD_FIFO_SIZE))
47         {
48             findMinMax(fifo_data2, AD_FIFO_SIZE, &vol_maxnum2, &vol_minnum2);
49             vol_amp2 = (vol_maxnum2 - vol_minnum2) * VOLTAGE_OFFSET / ADC_SCALE;
50         }
51     }
52 }
53
54 int ad_measure_phase_difference(float signal_freq, float calibration_ns,

```

Listing 24: ad_measure.c: 单/双通道 1024 点采集和 Vpp 更新

vpp_adc_parallel() 先分别配置有效通道，再等待完成标志。它名称中的“parallel”表示两路可在同一事务中采集，不表示 STM32 同时读两条 FMC 数据流；读回仍然按 AD1、AD2 顺序执行。

4.2 过零频率估计

阈值 h 位于样本 $x_{i-1} < h \leq x_i$ 之间时，线性插值过零位置为

$$p = i - 1 + \frac{h - x_{i-1}}{x_i - x_{i-1}}. \quad (4.3)$$

若共有 $K \geq 4$ 个同向过零点，首末位置为 p_0 、 p_{K-1} ，则

$$\hat{f}_{zc} = f_s \frac{K - 1}{p_{K-1} - p_0}. \quad (4.4)$$

```

1 static float estimateFrequency(const u16 *samples, int size, float sampling_freq)
2 {
3     u16 max_value = samples[0];
4     u16 min_value = samples[0];
5     for (int i = 1; i < size; i++)
6     {
7         if (samples[i] > max_value) max_value = samples[i];
8         if (samples[i] < min_value) min_value = samples[i];
9     }
10
11     u16 range = max_value - min_value;

```

```

12     if (range < ADC_FREQ_MIN_RANGE_COUNTS)
13     {
14         return 0.0f;
15     }
16
17     float midpoint = min_value + range * 0.5f;
18     float arm_level = min_value + range * 0.4f;
19     float first_crossing = 0.0f;
20     float last_crossing = 0.0f;
21     int crossing_count = 0;
22     int armed = samples[0] <= arm_level;
23
24     for (int i = 1; i < size; i++)
25     {
26         float previous = samples[i - 1];
27         float current = samples[i];
28
29         if (current <= arm_level)
30         {
31             armed = 1;
32         }
33         if (armed && previous < midpoint && current >= midpoint && current > previous)
34         {
35             float crossing = (i - 1) + (midpoint - previous) / (current - previous);
36             if (crossing_count == 0) first_crossing = crossing;
37             last_crossing = crossing;
38             crossing_count++;
39             armed = 0;
40         }
41     }
42
43     if (crossing_count < ADC_FREQ_MIN_CROSSINGS || last_crossing <= first_crossing)
44     {
45         return 0.0f;
46     }
47
48     float frequency = sampling_freq * (crossing_count - 1) /
49                     (last_crossing - first_crossing);
50     return (frequency <= sampling_freq * 0.25f) ? frequency : 0.0f;
51 }

```

Listing 25: ad_measure.c: 带迟滞、线性插值和最少跨越数的过零测频

算法不使用固定 0 V 阈值，而是从当前 128/1024 点的最大值和最小值计算中点。40% 量程位置作为重新布防阈值，只有信号先回到低于该阈值时，下一次上升跨越才会被计数。这可抑制中点附近噪声引起的重复计数。最终还要求至少 4 个同向过零点，并拒绝超过 $f_s/4$ 的结果。

4.3 Hann 窗、FFT 与幅值恢复

长度 $N = 1024$ 的 Hann 窗为

$$w[n] = \frac{1}{2} \left(1 - \cos \frac{2\pi n}{N-1} \right). \quad (4.5)$$

窗后 DFT 及相干增益为

$$X_w[k] = \sum_{n=0}^{N-1} x[n]w[n]e^{-j2\pi kn/N}, \quad G_c = \frac{1}{N} \sum_{n=0}^{N-1} w[n]. \quad (4.6)$$

实信号单边峰值幅度估计为

$$\hat{A}_k = \frac{2}{NG_c} |X_w[k]|. \quad (4.7)$$

4.3.1 CMSIS-DSP 缓冲组织与归一化

```

1 void calculate_fft_spectrum(const float *input_data, uint16_t data_length)
2 {
3     uint16_t i;
4     uint16_t actual_length = (data_length > FFT_LENGTH) ? FFT_LENGTH : data_length;
5     float mean = 0.0f;
6     float window_sum = 0.0f;
7
8     // 1. 清空缓冲区，确保每次计算的初始状态干净
9     memset(fft_input_buffer, 0, sizeof(fft_input_buffer));
10    memset(fft_magnitude, 0, sizeof(fft_magnitude));
11
12    for (i = 0; i < actual_length; i++) mean += input_data[i];
13    if (actual_length > 0U) mean /= actual_length;
14
15    // 2. Hann窗抑制非相干采样泄漏，幅度按窗函数相干增益补偿。
16    for (i = 0; i < actual_length; i++)
17    {
18        float window = spectrum_window[i];
19        fft_input_buffer[2*i] = (input_data[i] - mean) * window;
20        fft_input_buffer[2*i + 1] = 0.0f;
21        window_sum += window;
22    }
23    // 如果实际数据长度小于FFT_LENGTH，缓冲区剩余部分已清零，这相当于进行了“零填充(Zero Padding)”
24    // 零填充不提高频谱的物理分辨率，但可以增加频点密度，有助于观察和插值。
25
26    // 3. 执行核心的复数FFT计算
27    arm_cfft_radix4_f32(&fft_instance, fft_input_buffer);
28
29    // 4. 计算复数结果的模，得到幅度谱
30    arm_cmplx_mag_f32(fft_input_buffer, fft_magnitude, FFT_LENGTH);
31
32    // 5. Hann窗幅度归一化；直流分量保留采样均值。

```



```

33     fft_magnitude[0] = fabsf(mean);
34     for (i = 1; i < FFT_LENGTH / 2; i++)
35     {
36         fft_magnitude[i] = (window_sum > 0.0f)
37             ? fft_magnitude[i] * 2.0f / window_sum : 0.0f;
38     }
39 }
40
41 int fft_get_phase(const float *input_data, float sampling_freq,
42                 float signal_freq, float *phase_deg)
43 {
44     if (input_data == NULL || phase_deg == NULL || sampling_freq <= 0.0f ||
45         signal_freq <= 0.0f || signal_freq >= sampling_freq * 0.5f)
46         return 0;
47
48     uint16_t target_bin = (uint16_t)(signal_freq * FFT_LENGTH /
49                                     sampling_freq + 0.5f);
50     if (target_bin < 1U || target_bin >= FFT_LENGTH / 2U)
51         return 0;
52
53     calculate_fft_spectrum(input_data, FFT_LENGTH);
54     if (fft_magnitude[target_bin] < FFT_PHASE_MIN_MAGNITUDE)
55         return 0;
56
57     *phase_deg = atan2f(fft_input_buffer[2U * target_bin + 1U],
58                       fft_input_buffer[2U * target_bin]) *
59         (180.0f / 3.14159265f);
60     return 1;
61 }

```

Listing 26: my_fft.c: 去均值、Hann 加窗、复数 FFT 和幅值恢复

CMSIS 复数输入使用实部、虚部交错存放,因此第 i 个实数样本写入 `buffer[2*i]`, `buffer[2*i+1]` 置零。先减均值可降低直流分量向低频主瓣的泄漏。幅度谱的非直流频点乘 2 是因为只显示正频率半谱,再除以窗函数和则补偿 Hann 窗的相干增益。

零填充只增加显示频点密度,不增加由实际采集时长决定的物理分辨率。当需要更高频率准确度时,应优先延长有效观测时间或改进频率估计器,而不是只增加零填充点数。

4.3.2 三点抛物线峰值插值

```

1 float get_precise_peak_frequency(float sampling_freq)
2 {
3     uint16_t i;
4     float freq_resolution = sampling_freq / FFT_LENGTH;
5
6     // 1. 寻找最大幅度点 (谱峰) 的索引
7     float max_magnitude = 0.0f;
8     uint16_t max_index = 1;
9     for (i = 1; i < FFT_LENGTH / 2; i++)

```

```

10     {
11         if (fft_magnitude[i] > max_magnitude)
12         {
13             max_magnitude = fft_magnitude[i];
14             max_index = i;
15         }
16     }
17
18     // 如果峰值点在边界, 无法进行三点插值, 直接返回粗略频率
19     if (max_index <= 1 || max_index >= (FFT_LENGTH / 2 - 1))
20     {
21         return max_index * freq_resolution;
22     }
23
24     // 2. 三点抛物线插值
25     float y1 = fft_magnitude[max_index - 1]; // 左边点
26     float y2 = fft_magnitude[max_index];      // 中间峰值点
27     float y3 = fft_magnitude[max_index + 1]; // 右边点
28
29     // 计算抛物线顶点相对于中心点(max_index)的偏移量 delta
30     float delta = 0.5f * (y1 - y3) / (y1 - 2.0f * y2 + y3);
31
32     // 检查分母是否过小, 防止除以零
33     if (fabsf(y1 - 2.0f * y2 + y3) < 1e-10f)
34     {
35         delta = 0.0f;
36     }
37
38     // 限制偏移量范围, 确保结果合理
39     if (delta > 0.5f) delta = 0.5f;
40     if (delta < -0.5f) delta = -0.5f;
41
42     // 3. 插值估计峰值频率
43     float precise_freq = (max_index + delta) * freq_resolution;
44
45     return precise_freq;
46 }
47
48 /**
49  * @brief 计算总谐波失真 (THD)
50  * @details THD = sqrt((谐波功率)) / sqrt(基波功率) * 100%
51  *          统计2至15次、且低于奈奎斯特频率的Hann主瓣功率。
52  * @param fundamental_freq 基波频率
53  * @param sampling_freq 采样频率
54  * @return (float) THD值 (以百分比形式返回)

```

Listing 27: my_fft.c: 谱峰索引搜索和三点抛物线插值

对最大频点 k 及左右频点幅值 y_1, y_2, y_3 , 峰值偏移为

$$\delta = \frac{1}{2} \frac{y_1 - y_3}{y_1 - 2y_2 + y_3}, \quad \hat{f} = (k + \delta) \frac{f_s}{N}. \quad (4.8)$$

代码将 δ 限制到 $[-0.5, 0.5]$, 并在分母过小或峰值位于边界时退化为频点中心。

4.4 双路基波相位

对第 c 路样本去直流后, 在已知基波 f_0 上计算

$$I_c = \sum_{n=0}^{N-1} (x_c[n] - \bar{x}_c) \cos \left(2\pi \frac{f_0}{f_s} n \right), \quad (4.9)$$

$$Q_c = - \sum_{n=0}^{N-1} (x_c[n] - \bar{x}_c) \sin \left(2\pi \frac{f_0}{f_s} n \right), \quad (4.10)$$

$$\phi_c = \text{atan2}(Q_c, I_c). \quad (4.11)$$

AD2 同序号样本相对 AD1 延后

$$\Delta t = \frac{1}{2f_{s,actual}} + t_{cal}. \quad (4.12)$$

最终物理相位差定义为

$$\Delta\phi_{2-1} = \text{wrap}_{(-\pi, \pi]}(\phi_2 - \phi_1 - 2\pi f_0 \Delta t). \quad (4.13)$$

4.4.1 双路采集与相位补偿代码

```

1          float *phase_difference_deg)
2  {
3      float phase1;
4      float phase2;
5
6      if (signal_freq <= 0.0f || phase_difference_deg == NULL)
7          return 0;
8
9      vol_amp1 = 0.0f;
10     vol_amp2 = 0.0f;
11     setShortCaptureMode(1, 0);
12     setShortCaptureMode(2, 0);
13     setSamplingFrequency(signal_freq, 1, FREQ_MODE_SIGNAL);
14     setSamplingFrequency(signal_freq, 2, FREQ_MODE_SIGNAL);
15     CTRL_DATA = CTRL_DATA | AD_SYNC_MODE;
16     resetFifoPair();
17     AD_FIFO_WRITE_ENABLE(3);
18     HAL_Delay(1);
19
20     while (AD1_FULL_FLAG != 0x0001 || AD2_FULL_FLAG != 0x0001)
21         ;

```

```

22
23 CTRL_DATA = CTRL_DATA & ~(AD1_FIFO_WR | AD2_FIFO_WR));
24 int valid1 = readFIFOData(1, fifo_data1, fifo_data1_f, AD_FIFO_SIZE);
25 int valid2 = readFIFOData(2, fifo_data2, fifo_data2_f, AD_FIFO_SIZE);
26 CTRL_DATA = CTRL_DATA & (~AD_SYNC_MODE);
27 if (!valid1 || !valid2)
28     return 0;
29
30 findMinMax(fifo_data1, AD_FIFO_SIZE, &vol_maxnum1, &vol_minnum1);
31 findMinMax(fifo_data2, AD_FIFO_SIZE, &vol_maxnum2, &vol_minnum2);
32 vol_amp1 = (vol_maxnum1 - vol_minnum1) * VOLTAGE_OFFSET / ADC_SCALE;
33 vol_amp2 = (vol_maxnum2 - vol_minnum2) * VOLTAGE_OFFSET / ADC_SCALE;
34
35 if (!fft_get_phase(fifo_data1_f, adc_fft_sampling_freq[0], signal_freq,
36                   &phase1))
37     return 0;
38 if (!fft_get_phase(fifo_data2_f, adc_fft_sampling_freq[0], signal_freq,
39                   &phase2))
40     return 0;
41
42 // 两个NCO同步装载0°/180°; 同序号AD2样本比AD1晚半个实际采样周期。
43 float sampling_skew_deg = signal_freq * 180.0f / adc_fft_sampling_freq[0];
44 float calibration_deg = calibration_ns * 1e-9f * signal_freq * 360.0f;
45 *phase_difference_deg = phase2 - phase1 - sampling_skew_deg - calibration_deg;
46 while (*phase_difference_deg > 180.0f) *phase_difference_deg -= 360.0f;
47 while (*phase_difference_deg <= -180.0f) *phase_difference_deg += 360.0f;
48 return 1;
49 }

```

Listing 28: ad_measure.c: 同步双 FIFO 采集、基波相位和半采样周期补偿

当前输出定义为 AD2-AD1。由于同序号 AD2 样本比 AD1 晚半个实际采样周期，其等效相位延迟是

$$\phi_{skew} = 360^\circ f_0 \frac{1}{2f_s} = 180^\circ \frac{f_0}{f_s}. \quad (4.14)$$

代码还保留 `calibration_ns` 板级固定时差校准项。该参数只能补偿经实测确认的固定延迟，不能用来抵消 FIFO 错位、通道增益错误或频率估计错误。

`fft_get_phase()` 只在目标频率位于 $(0, f_s/2)$ 、目标频点有效且幅值高于 0.02 V 时返回相位。因此「invalid」是正常错误路径，调用者不应把失败结果当作 0°。

4.5 THD、THD+N 与 SINAD

设基波主瓣功率为 P_1 ，第 h 次谐波主瓣功率为 P_h ，除 DC 和基波主瓣外的频内功率为 P_{nd} ：

$$\text{THD} = 100\% \sqrt{\frac{\sum_{h=2}^{15} P_h}{P_1}}, \quad (4.15)$$

$$\text{THD} + \text{N} = 100\% \sqrt{\frac{P_{nd}}{P_1}}, \quad (4.16)$$

$$\text{SINAD} = 10 \log_{10} \frac{P_1}{P_{nd}} \text{ dB}. \quad (4.17)$$

```

1 float calculate_thd(float fundamental_freq, float sampling_freq)
2 {
3     float freq_resolution = sampling_freq / FFT_LENGTH;
4     float fundamental_power;
5     float harmonic_power = 0.0f;
6
7     uint16_t fundamental_bin = (uint16_t)(fundamental_freq / freq_resolution + 0.5f);
8     if (fundamental_bin < 1 || fundamental_bin >= FFT_LENGTH / 2)
9     {
10         return 0.0f;
11     }
12
13     fundamental_power = sumSpectrumBandPower(fundamental_bin);
14
15     // 累计每次谐波的完整Hann主瓣，避免频率偏差造成单频点漏算。
16     for (uint8_t harmonic = 2; harmonic <= 15; harmonic++)
17     {
18         uint16_t harmonic_bin = (uint16_t)(harmonic * fundamental_freq / freq_resolution + 0.5f)
19         ;
20         if (harmonic_bin >= FFT_LENGTH / 2)
21         {
22             break;
23         }
24
25         harmonic_power += sumSpectrumBandPower(harmonic_bin);
26     }
27
28     if (fundamental_power > 1e-20f)
29     {
30         return sqrtf(harmonic_power / fundamental_power) * 100.0f;
31     }
32
33     return 0.0f;
34 }
35 /**
36  * @brief 计算THD+N（总谐波失真加噪声）
37  * @details THD+N = sqrt((总功率 - 基波功率)) / sqrt(基波功率) × 100%
38  * 包含了谐波失真和噪声的综合指标，更全面地评估信号质量

```

```

39  * @param fundamental_freq 基波频率
40  * @param sampling_freq 采样频率
41  * @return (float) THD+N值 (以百分比形式返回)
42  */
43 float calculate_thd_n(float fundamental_freq, float sampling_freq)
44 {
45     float freq_resolution = sampling_freq / FFT_LENGTH;
46     float fundamental_power;
47     float total_power = 0.0f;
48
49     uint16_t fundamental_bin = (uint16_t)(fundamental_freq / freq_resolution + 0.5f);
50     if (fundamental_bin < 1 || fundamental_bin >= FFT_LENGTH / 2)
51     {
52         return 0.0f;
53     }
54
55     fundamental_power = sumSpectrumBandPower(fundamental_bin);
56
57     // 总功率排除直流，再扣除基波完整Hann主瓣。
58     for (uint16_t i = 1; i < FFT_LENGTH / 2; i++)
59     {
60         total_power += fft_magnitude[i] * fft_magnitude[i];
61     }
62
63     float distortion_noise_power = total_power - fundamental_power;
64     if (fundamental_power > 0.0f && distortion_noise_power >= 0.0f)
65     {
66         return sqrtf(distortion_noise_power / fundamental_power) * 100.0f;
67     }
68
69     return 0.0f;
70 }
71
72 /**
73  * @brief 计算信纳比(SINAD - Signal to Noise and Distortion Ratio)
74  * @details SINAD = 基波功率 / (噪声功率 + 失真功率) (dB)
75  *          SINAD是THD+N的倒数，以dB形式表示信号质量
76  * @param fundamental_freq 基波频率
77  * @param sampling_freq 采样频率
78  * @return (float) SINAD值 (以dB形式返回)
79  */
80 float calculate_sinad(float fundamental_freq, float sampling_freq)
81 {
82     float thd_n_percent = calculate_thd_n(fundamental_freq, sampling_freq);
83
84     if (thd_n_percent > 0.0f)
85     {
86         // SINAD(dB) = -20*log10(THD+N/100)
87         float sinad_db = -20.0f * log10f(thd_n_percent / 100.0f);
88         return sinad_db;

```

```

89     }
90
91     return 0.0f;

```

Listing 29: my_fft.c: Hann 主瓣功率累加、THD、THD+N 与 SINAD

单个 Hann 窗正弦的能量不只落在一个频点。当信号频率不完全对齐 FFT 频点时，若只累加中心频点会把主瓣的其余能量错当噪声。当前实现对每个基波/谐波中心累加 ± 2 个频点，并使用 Hann 等效噪声带宽 1.5 对显示幅值进行近似恢复。

该指标仍受以下条件限制：

- 1024 点有限观测时间和采样率决定真实频率分辨率；
- AD9226 前端噪声、板级时钟抖动和量化噪声都进入 THD+N；
- 超过奈奎斯特频率的谐波不会按原频率计算，而是可能折叠回带内；
- 未进行多量程、多频率、温度和多板统计时，不应将单次输出当作整机保证值。

5 接口与引脚摘要

5.1 FPGA 外部接口

表 6: FPGA 主要外部接口

接口	位宽/数量	说明
FPGA_DB	16 位	FMC 复用地址/数据总线
AD1/2_INPUT	各 12 位	双路 AD9226 并行数据
AD1/2_OUTCLK	各 1 位	双路 ADC 采样输出时钟
AD1/2_INPUT_CLK	各 1 位	外部比较器测频输入
DA1/2_OUT	各 14 位	双路 AD9764 数据
DA1/2_OUTCLK	各 1 位	双路 AD9764 采样时钟

5.2 STM32 外设接口

STM32 通过 PD/PE 端口提供 FMC D0–D15，通过 PB7、PG9、PD4、PD5 提供地址锁存、片选、读和写控制。AD9959 使用 P0–P3、SDIO0–SDIO3、SCK、CS、RESET、PDC 和 IO_UPDATE；两片 AD9833 分别使用 FSYNC、SCLK 和 SDATA。完整封装引脚见 硬件接口与寄存器映射.md。

6 构建与验证

6.1 构建环境

- FPGA: Quartus Prime 18.1, EP4CE10F17C8, Standard Fit。

- STM32: Keil MDK, ARM Compiler 5.06 update 7。
- 文档: 在仓库根目录运行 `build_latex.ps1`, 使用 XeLaTeX 生成 docs/zuolan_v5 系统设计说明.pdf。编译过程使用系统临时目录, 完成后自动清理辅助文件。

最近一次完整 FPGA 构建为 0 error、9 warning、1,379 个逻辑单元, 全局最差 setup 为 +0.037ns, hold 为 +0.148ns。最近一次 STM32 构建为 0 error、2 warning, Code 为 33,592 bytes。

6.2 轻量回归

```
python zuolan_stm32/test/test_dual_adc_phase.py
python zuolan_stm32/test/test_key4_measurement_modes.py
python zuolan_stm32/test/test_adaptive_frequency.py
python zuolan_stm32/test/test_fft_thd.py
python zuolan_stm32/test/test_fifo_reset_contract.py
pwsh -NoProfile -ExecutionPolicy Bypass \
-File zuolan_FPGA/script/check_pure_verilog.ps1
```

实板已使用 5kHz、约 1Vpp 的同相、90° 和 180° 输入确认双路幅度稳定在约 1V, 输出相位随输入变化正确。该结果验证当前采样配对路径, 不代表未经扫频、量程、温度与多板统计的整机精度指标。

6.3 验证层次与证据边界

表 7: 当前可用的验证证据

层次	可证明内容	不能外推的内容
静态检查	纯 Verilog 约束、地址/控制位一致性	不证明板级建立/保持时间或模拟前端性能。
Python 回归	双路相位符号、采样率策略、THD 功率累加和 FIFO 控制契约	不使用真实 ADC/FMC 电气时序。
Quartus/Keil 构建	HDL/C 语法、资源、时序分析和链接完整性	不代替上板后的通道校准和连线检查。
实板定点测试	5kHz、1Vpp、0/90/180° 附近的幅度与相位路径	不证明全频带、全量程、温度漂移或多板一致性。

6.4 常见故障与定位顺序

表 8: 常见现象、直接证据与检查路径

现象	优先证据	检查顺序
FIFO 一直未完成	Bit4/6、Bit12/13、Bit14/15 和 采样时钟引脚	先确认 NCO 使能，再确认 FIFO 已离开复位，最后检查采集点数模式。
读数高 4 位非零	原始 FMC 16 位读值	检查 0x06/0x08 直读旁路、FMC 读时序、FIFO 读时钟和数据线顺序，禁止用掩码修复。
AD2 幅度偶发异常	双 FIFO 原始码范围、读取顺序	先排查异步 FMC 混合字，再检查通道引脚和模拟前端，不先改 Vpp 算法。
相位固定差 180°	信号源接线、AD1/AD2 标签、初相装载	确认输出定义 AD2-AD1，再检查 0/180° NCO 初相与半周期补偿符号。
FFT 主峰两侧对称抬高	相干性、Hann 窗和采样率	对非相干采样这是正常主瓣；确认幅值恢复使用窗函数和，不把相邻主瓣当谐波。
THD+N 明显高于 THD	全带宽噪声、主瓣扣除范围	这通常是正常定义差异；再检查采样时钟杂散、前端噪声和谐波折叠。
Key4 影响调制更新	任务执行时间和 USART 输出量	当前调度器非抢占；先减少频谱输出，再考虑将采集/输出改为分步状态机。

6.5 日志解读

一次正常 Key4 测量日志应按顺序包含：信号频率、Vpp、双路模式下的 AD2-AD1 相位、物理采样率、FFT 频率分辨率、峰值频率、THD、THD+N、SINAD 和 DC。如果日志在「Calculating FFT」之前停止，优先检查 FIFO 完成标志；如果频谱输出完整但相位 invalid，优先检查目标频点幅值和采样率。

7 结论与维护约束

当前设计通过双 NCO 同步初相装载和固定 FIFO 样本配对，避免了组合时钟复用导致的 AD2 跳码。STM32 使用实际 NCO 采样率补偿半采样周期，并保留板级固定时差校准项。

后续修改必须遵守以下约束：

- 1) FMC 地址或控制位变化时，同步修改 FPGA、STM32、测试与文档。
- 2) 采样时钟、初相或写入延迟变化时，重新验证双路配对和相位补偿符号。
- 3) 不使用限幅、鲁棒 Vpp 或比例修正掩盖非法 FIFO 数据。

- 4) 保留作者“左岚”；联合开发文件保留原联合作者。
- 5) 未经校准和统计验证，不声明固定整机精度、动态范围或 THD 下限。

A Key4 默认配置

```
#define KEY4_FREQ_SOURCE KEY4_FREQ_SOURCE_ADC_AUTO
#define KEY4_MANUAL_SIGNAL_FREQ 5000.0f
#define KEY4_RUN_FFT 1
#define KEY4_MEASURE_MODE KEY4_MODE_PHASE_PAIR
#define KEY4_PHASE_CALIBRATION_NS 0.0f
```

B 术语与符号表

表 9: 本文档的统一术语

术语/符号	定义
采样时钟	FPGA NCO 输往 AD9226 的方波，不是 FIFO 的 wrclk。
采样沿	采样时钟上升沿，在 clk_base 域中被检测为单周期脉冲。
写请求	TYFIFO.wrreq，在检测到采样沿后的下一个 150 MHz 周期生效。
同序号样本	AD1 FIFO 和 AD2 FIFO 中索引相同的样本；同步模式下 AD2 物理采样时刻晚 $T_s/2$ 。
物理采样率	NCO 频率字量化后的实际输出频率，用于相位时差补偿和 FFT 频轴。
等效 FFT 采样率	当数据组代表特定周期布局时的频轴参数；当前主流程与物理采样率一致。
固定配对	通过同步初相、AD1 起始门控和独立 FIFO 保证双路索引关系不随启动时刻变化。
M	32 位 NCO 频率字。
T_s	ADC 物理采样周期。
T_{clk}	clk_base 周期，当前为 1/150 MHz。
P_1, P_h, P_{nd}	基波主瓣功率、第 h 次谐波主瓣功率、除 DC 和基波外的频内功率。

C 关键源码索引

表 10: 代码阅读路线

文件	关键入口	推荐阅读目的
zuolan_FPGA/src/TOP.v	边沿检测、实例化	理解端口、时钟、FMC 映射和双 FIFO 配对。
zuolan_FPGA/src/FMC_CONTROLLER.v	地址锁存、直读旁路	理解异步 FMC 事务与 FIFO 读回契约。
zuolan_FPGA/src/FREQ_DEV.v	32 位累加器	理解采样率频率字和同步初相。
zuolan_FPGA/src/DA_*	相位累加、ROM 选择	理解双 DA 频率、相位和波形生成。
VOLTAGE_SCALER_CLOCKED.v	Q24 流水	理解幅度限制和 DAC 数据边沿。
key_app.c	key4_action_perform	从用户操作进入完整测量流程。
ad_measure.c	setSamplingFrequency	理解采样、FIFO、Vpp、自动测频和相位。
	ad_detect_frequency	
	ad_measure_phase_difference	
freq_measure.c	fre_measure	理解比较器门控测频。
da_output.c	DA_Apply_Settings	理解软件影子配置与 FPGA DDS 原子提交。
my_fft.c	calculate_fft_spectrum	理解窗函数、幅值、频率、相位和失真指标。
commond_init.h	控制位与地址宏	理解 STM32/FPGA 跨层契约。

D 相关文档

- docs/总体模块讲解.md
- docs/FPGA 模块使用说明.md
- docs/STM32 模块使用说明.md
- docs/硬件接口与寄存器映射.md